



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Computer Vision

Computer Vision (S2-25_AIMLZG525)

Session #3:
Low Level Vision

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snaveley

Raghavendra Singh



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

- Image Histogram & Histogram Equalization
- Basic Intensity Transformations
- Image Smoothing

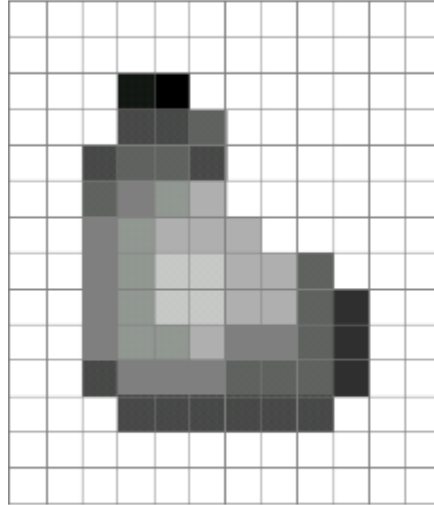
Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely



What is an image?

A grid (matrix) of intensity values

0	Absolute Black
1 - 127	Dark Grays
128	Medium Gray
129 - 254	Light Grays
255	Absolute White



255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255
255	255	127	145	145	175	127	127	95	47	255	255
255	255	74	127	127	127	95	95	95	47	255	255
255	255	255	74	74	74	74	74	74	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255

The slide defines an image as "**A grid (matrix) of intensity values.**" * **Grid/Matrix:** Images are divided into a rows-and-columns structure. Each individual square in this grid is a **pixel** (picture element).

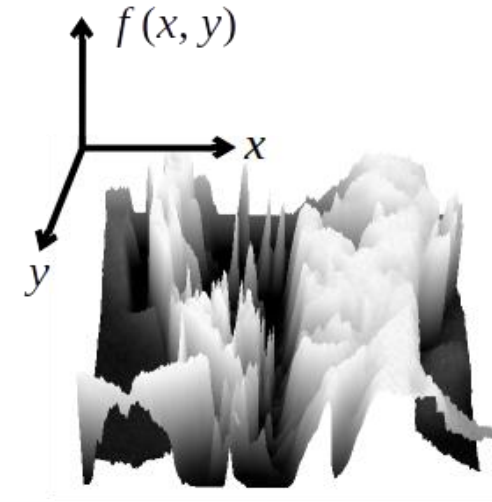
- **Intensity Values:** Each pixel is assigned a numerical value that represents how bright or dark that specific spot is.

What is an image?

Can think of a (grayscale) image as a function f from $\mathbb{R}^2$ to $\mathbb{R}$:
 $f(x,y)$ gives the intensity at position (x,y)



[snoop](#)



[3D view](#)

The text explains that a grayscale image can be defined as a function f that maps coordinates to a specific value:

- **Domain ($\mathbb{R}^2$):** The 2D space of the image, defined by (x, y) coordinates (width and height).
- **Range ($\mathbb{R}$):** The intensity or "brightness" value at that specific point.

In simple terms:

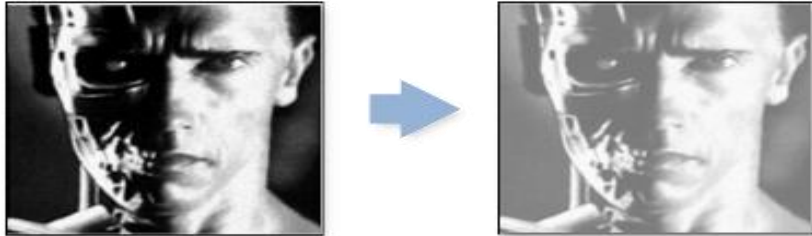
$$f(x, y) = \text{Intensity}$$

- **Dark areas** (like the shadows) have low intensity values (close to 0).
- **Bright areas** (like the highlights on the face) have high intensity values.



Image Transformations

As with any function, we can apply operators to an image



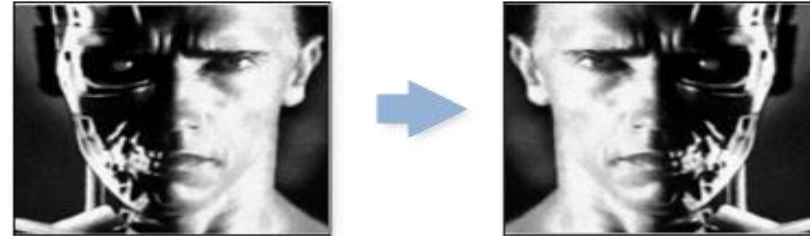
$$g(x,y) = f(x,y) + 20$$

1. Intensity Transformation (Brightness)

The first example shows an operation on the **output** (intensity) of the function.

$$g(x,y) = f(x,y) + 20$$

- **What is happening:** This adds a constant value (20) to the brightness of every single pixel.
- **The result:** The image becomes **brighter** overall. You can see in the comparison that the dark shadows become gray and the light areas become even whiter.



$$g(x,y) = f(-x,y)$$

2. Geometric Transformation (Mirroring)

The second example shows an operation on the **input** (coordinates) of the function.

$$g(x,y) = f(-x,y)$$

- **What is happening:** This negates the x-coordinate. Instead of looking up the pixel value at x , the computer looks up the value at $-x$.
- **The result:** The image is **flipped horizontally** (mirrored) across the vertical axis. Notice how the Terminator's robotic eye moves from the left side of the frame to the right.

Transformation Type	Math Operation	Visual Effect
Intensity	Adding to $f(x,y)$	Changes brightness/contrast
Geometric	Changing (x,y)	Changes position/orientation



Image Histogram

Un-normalized histogram: $h(r_k) = n_k$ for $k = 0, 1, 2, \dots, L - 1$

Note: n_k is the number of pixels in f with intensity r_k

Normalized histogram: $p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN}$

Note: M and N are the number of image rows and columns

The sum of $p(r_k)$ for all values of k is always 1

Mostly, we work with normalized histograms



1. Un-normalized Histogram

The un-normalized histogram is a simple count of how many pixels in an image have a specific brightness level.

- **Formula:**

$$h(r_k) = n_k$$

- r_k : Represents the k^{th} intensity level (e.g., in an 8-bit image, values range from 0 to 255).
- n_k : The total number of pixels in the image that have the intensity r_k .
- **Context:** If you have a 100×100 image, the sum of all n_k values will be 10,000.

2. Normalized Histogram

A normalized histogram transforms these counts into **probabilities**. This is much more useful because it allows you to compare images of different sizes.

- **Formula:**

$$p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN}$$

- **M and N :** The dimensions of the image (rows and columns). MN is the total number of pixels.
- **Key Property:** The sum of all $p(r_k)$ values is always 1.
- **Why use it?** It gives you a statistical view of the image. For example, if $p(r_k) = 0.15$, it means 15% of the image consists of pixels with intensity r_k .

The slide notes that we "**Mostly work with normalized histograms.**" This is because:

1. **Scale Invariance:** You can compare a tiny thumbnail to a high-resolution photo; their normalized histograms will look similar if the content is the same.
2. **Probability Theory:** It allows us to apply mathematical techniques like **Histogram Equalization** to improve image contrast.

Summary: An un-normalized histogram tells you "how many," while a normalized histogram tells you "what percentage."

Elementary Intensity Transformations

$$s = L - 1 - r$$

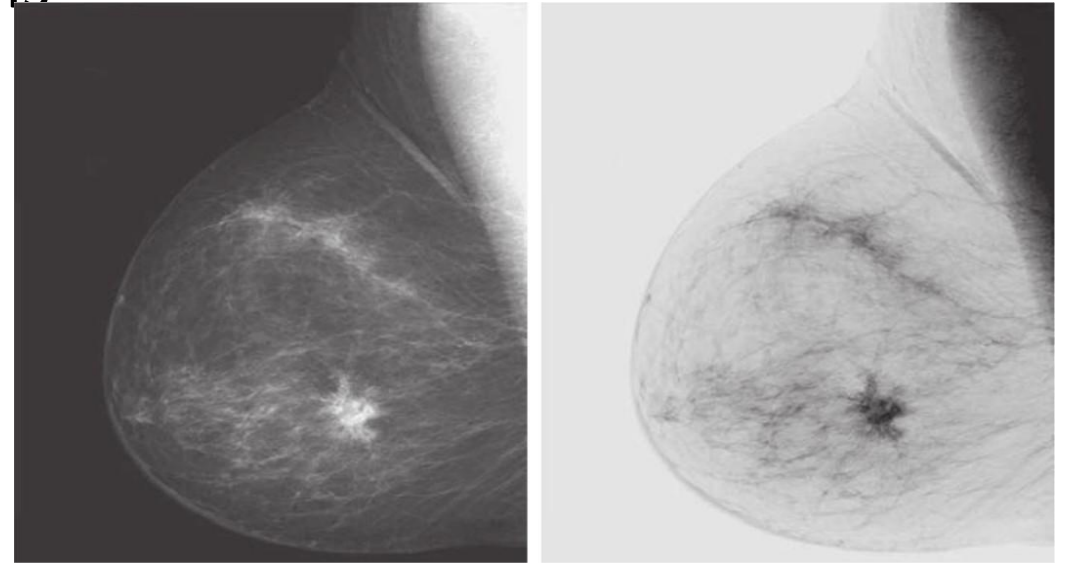
The Core Concept: Image Negatives

The transformation works by reversing the intensity levels of an image, similar to a photographic negative. This is particularly useful in medical imaging to enhance white details against large dark areas.

The process is governed by the following equation:

$$s = L - 1 - r$$

- r : The original pixel intensity (input).
- s : The transformed pixel intensity (output).
- L : The total number of possible intensity levels in the image.
For a standard 8-bit image, $L = 256$, making the formula $s = 255 - r$.



A digital mammogram and its Negative image

- **Original Image (Left):** Shows the dense tissue and potential abnormalities as light areas against a dark background.
- **Negative Image (Right):** By applying the transformation, the dark background becomes light, and the internal structures (like the lesions or dense tissue) become dark.
- **Purpose:** This inversion can make certain structural details much easier for a radiologist to analyze visually.



Elementary Intensity Transformations

Log Transformations

Fourier spectrum displayed as a grayscale image and the result of applying the log transformation in with $c = 1$.

Both images are scaled to the range $[0, 255]$.

The Core Concept: Dynamic Range Compression

A log transformation is primarily used to expand the values of dark pixels in an image while compressing the higher-level values. This is essential when the dynamic range of the data (the spread between the lowest and highest values) is too large for a standard display to show effectively.

The transformation is defined by the following equation:

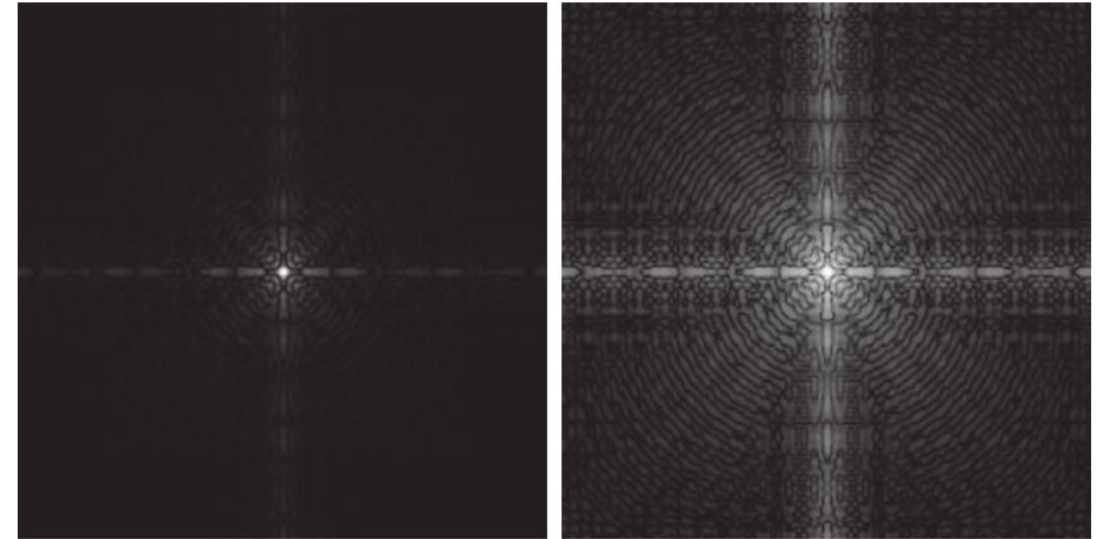
$$s = c \log(1 + r)$$

- r : The input pixel intensity. Adding 1 to r ensures that the log of zero (which is undefined) is avoided, as $\log(1) = 0$.
- s : The output (transformed) pixel intensity.
- c : A scaling constant used to adjust the output to a specific range.

Summary of Use

While the **Negative transformation** from the previous slide flips light and dark, the **Log transformation** pulls detail out of the shadows.

$$s = c \log(1 + r)$$



The slide demonstrates the power of this transformation using a **Fourier spectrum**:

- **Original Image (Left)**: The original spectrum has a massive dynamic range. Most of the important detail is "hidden" in the dark areas because the central peak is so much brighter than everything else.
- **Transformed Image (Right)**: By applying the log transformation (using $c = 1$ in this example), the low-intensity details become visible. This allows the viewer to see the complex patterns in the spectrum that were previously too dark to detect.
- **Scaling**: To make them viewable on a screen, both images are scaled to the standard 8-bit range of **[0, 255]**.



Elementary Intensity Transformations

Elementary Intensity Transformations, which are basic mathematical operations used to modify the brightness and contrast of digital images.

Overview of Transformation Functions

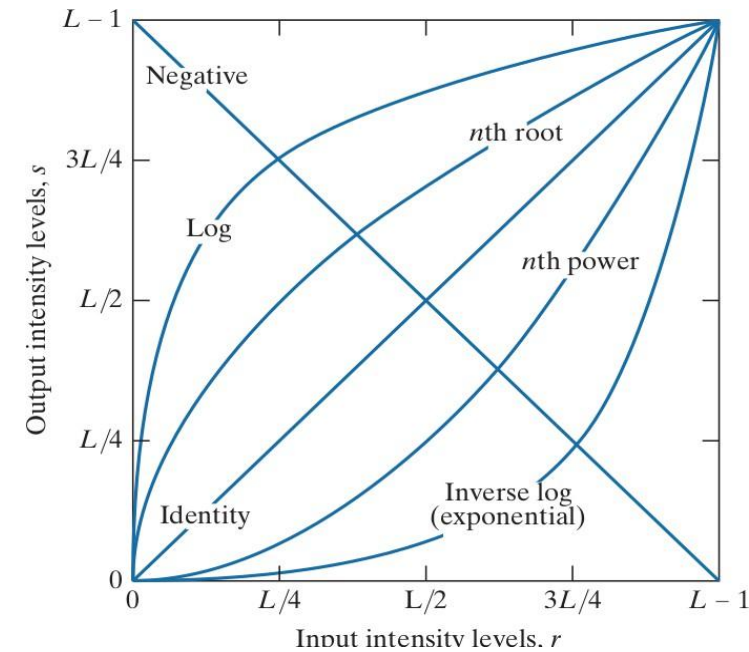
The graph in the final slide acts as a "cheat sheet" for how different mathematical functions map input pixel intensities (r) to output intensities (s).

- **Negative:** A linear transformation that flips the image (black becomes white).
- **Identity:** A direct 1-to-1 mapping where the output is exactly the same as the input.
- **Log:** Significantly expands the darker (low) values while compressing the higher ones.
- **Inverse Log:** The opposite of log; it compresses dark values and expands light ones.
- n^{th} root and n^{th} power: Non-linear curves (often called Gamma transformations) that shift the image balance toward lighter or darker tones.

1. Image Negative

This transformation reverses the intensity levels of an image.

- **Formula:** $s = L - 1 - r$.
- **Application:** In the provided mammogram example, the negative image makes the dense, light-colored tissue (potential abnormalities) appear as dark structures on a light background. This often makes it easier for the human eye to detect subtle patterns in a large dark area.



2. Log Transformation

This is used to visualize images with a high **dynamic range** (where some values are vastly larger than others).

- **Formula:** $s = c \log(1 + r)$.
- **Example (Fourier Spectrum):** In a Fourier spectrum, a few pixels might have extremely high intensities while others are very low.
 - **Without transformation:** The dark areas look completely black.
 - **With transformation:** The log function "pulls up" the dark values, revealing the hidden detail in the spectrum.
- **Scaling:** To view the results, the data is usually scaled back to a standard range like $[0, 255]$.



Elementary Intensity Transformations

Power-Law Transformations

$$s = cr^\gamma$$

- c and γ are positive constants

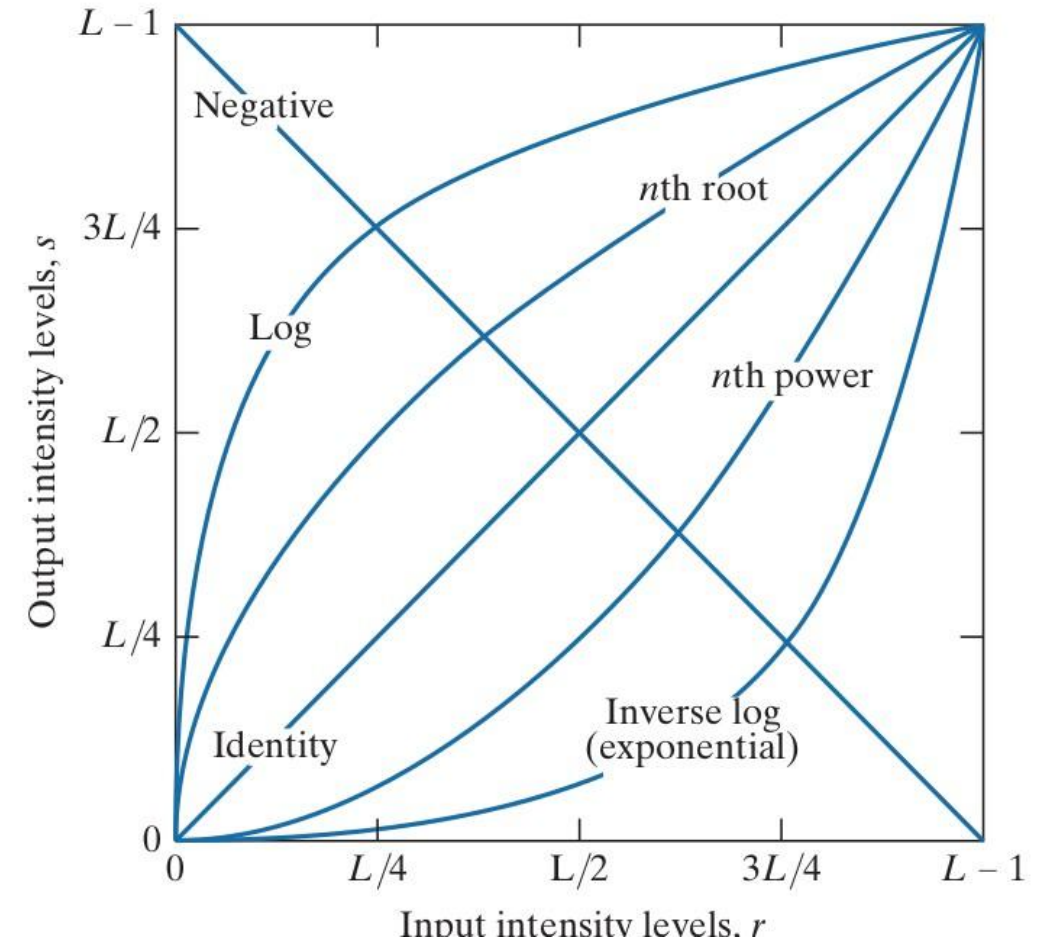
4. Power-Law (Gamma) Transformations

The final slide introduces **Power-Law Transformations**, also known as Gamma correction.

- **Formula:**

$$s = cr^\gamma$$

- c and γ (gamma) are positive constants.
- **Behavior:**
 - If $\gamma < 1$, the transformation behaves like a log function (brightening dark areas).
 - If $\gamma > 1$, it behaves like an inverse log function (darkening the image).





Elementary Intensity Transformations

Power-Law Transformations- Applications [Gamma correction](#)

- Cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with exponents varying from approximately 1.8 to 2.5.

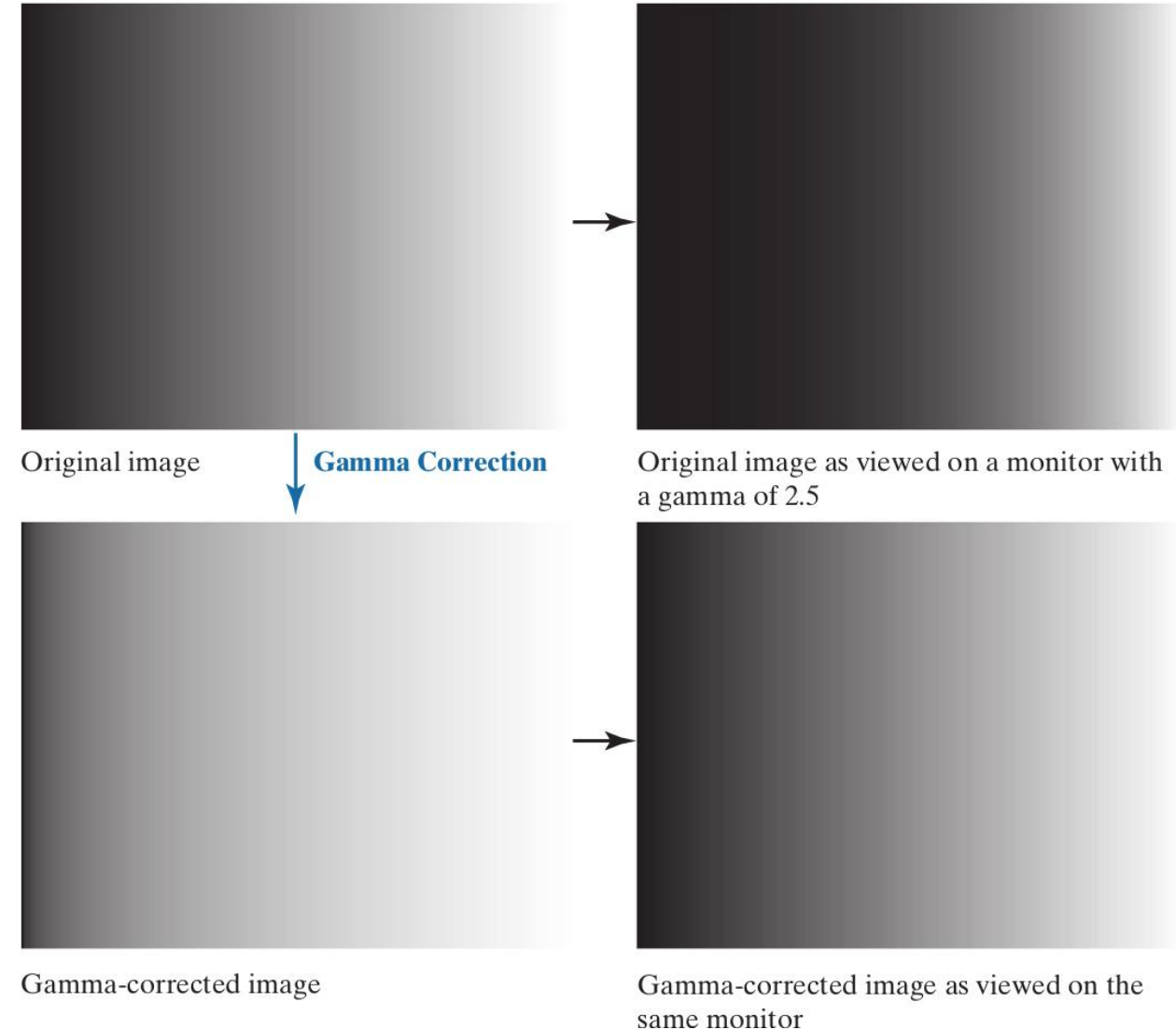
The Core Problem: Hardware Distortion

Most display devices (historically Cathode Ray Tubes or CRTs, but the logic still applies to modern monitors) do not have a linear relationship between voltage and brightness. Instead, they follow a **Power-Law** response.

As the slide notes, these devices have an exponent (gamma) typically between **1.8 and 2.5**. This means:

- If you send a medium-grey signal to the monitor, the monitor displays it as a **much darker grey**.
- This results in an image that looks "washed out" or overly dark in the shadows (as seen in the top-right image of the slide).

$$s = cr^\gamma$$

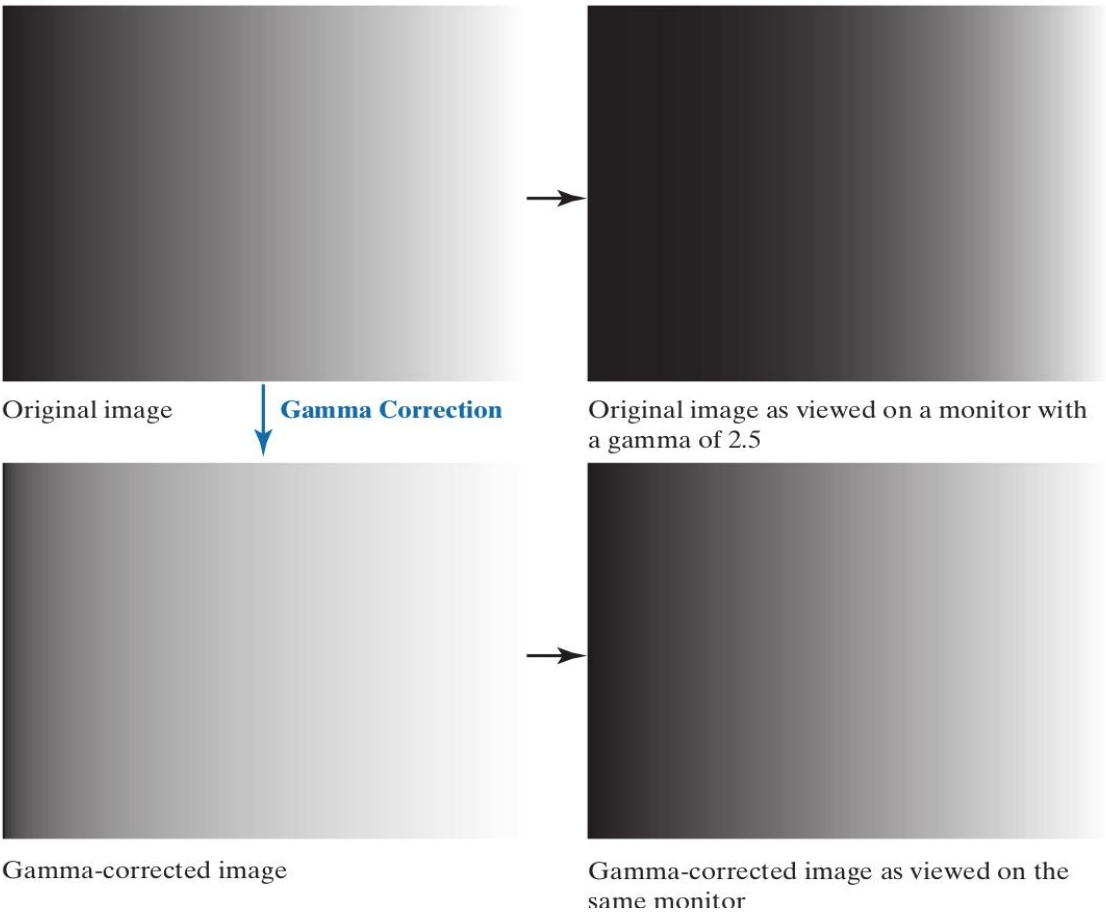




How Gamma Correction Works (The Process)

The images on the right side of the slide demonstrate the "fix":

- 1. **Top Left (Original Image):** This is the digital file as it exists in the computer.
- 2. **Top Right (Viewed on Monitor):** Because the monitor has a $\gamma = 2.5$, it naturally darkens the image. It looks muddy and dark.
- 3. **Bottom Left (Gamma-Corrected Image):** To fix this, we apply an **inverse** transformation to the original image *before* sending it to the monitor. We brighten the image artificially (this image looks "washed out" on a perfect screen).
- 4. **Bottom Right (Final Result):** When the "brightened" image is sent to the "darkening" monitor, the two effects **cancel each other out**. The result is a display that looks exactly like the original intended image.



Stage	Action	Visual Effect
No Correction	Image → Monitor	Image appears too dark/shadowy.
Gamma Correction	Image → Pre-brighten ($1/\gamma$)	Image looks washed out/bright.
Final Result	Pre-brightened Image → Monitor	Image looks natural and balanced.



Elementary Intensity Transformations

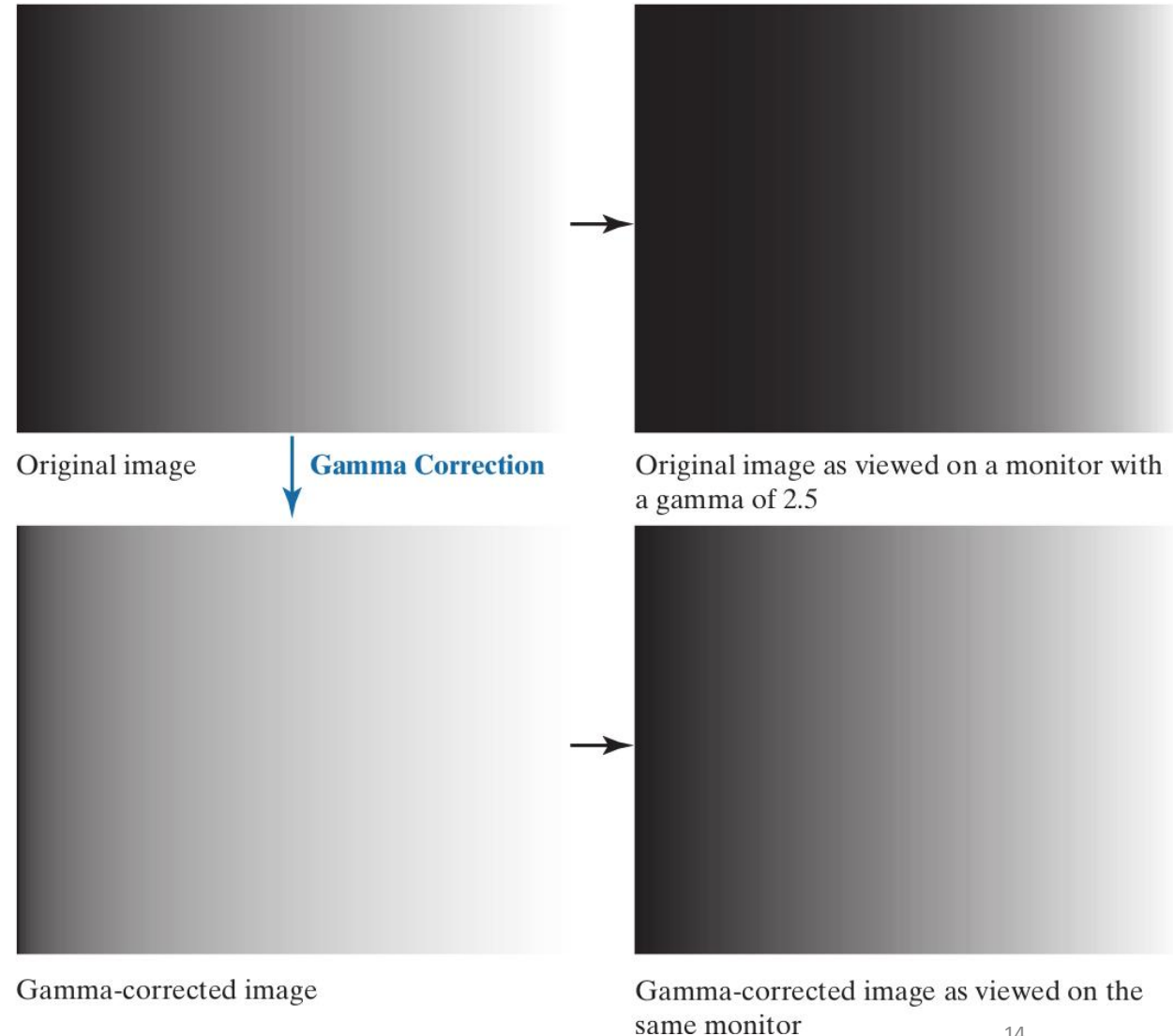
$$s = cr^\gamma$$

Power-Law Transformations- Applications

Gamma correction

- Cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with exponents varying from approximately 1.8 to 2.5.
- Gamma correction consists of using the transformation

$$s = r^{1/2.5} = r^{0.4}$$





The Technical Problem: CRT Response

- **Hardware Limitation:** Older display devices, like Cathode Ray Tubes (CRT), do not have a linear relationship between the voltage applied and the brightness produced.
- **Power Function:** These devices respond to intensity signals based on a power function, with exponents (γ) typically ranging from **1.8 to 2.5**.
- **Resulting Distortion:** Because of this hardware behavior, a standard image will appear significantly **darker** than intended when viewed on a monitor with a gamma of 2.5 (as shown in the top-right image of the slide).

The Solution: Gamma Correction

To counteract the monitor's darkening effect, a transformation is applied to the image *before* it is displayed.

- **The Math:** If a monitor has a gamma of 2.5, we apply a "correction" using the inverse of that value ($1/2.5 = 0.4$).
- **The Formula:** The transformation used is $s = r^{1/2.5} = r^{0.4}$.
- **The Effect:** This creates a **Gamma-corrected image** (bottom-left) that looks artificially bright or "washed out".

Final Outcome

The slide illustrates a four-step cycle:

1. **Original Image:** The intended visual.
2. **Uncorrected Viewing:** How the image would look (too dark) on a monitor with $\gamma = 2.5$.
3. **Gamma Correction:** Applying the $r^{0.4}$ transformation to the original image.
4. **Corrected Viewing:** When the pre-brightened image is processed by the monitor's natural darkening effect, the two cancel each other out, and the image appears correct to the human eye (bottom-right).



Elementary Intensity Transformations

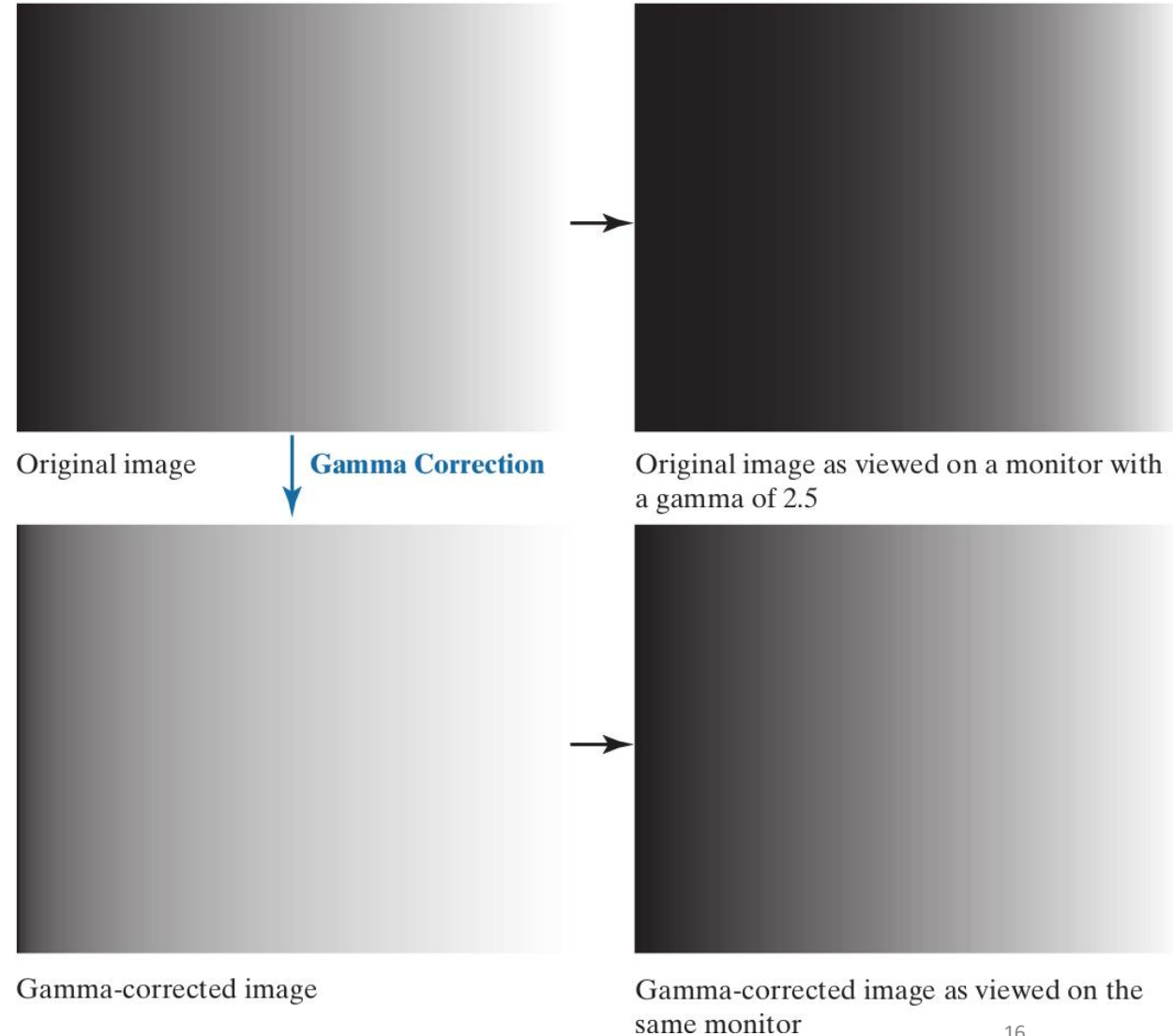
Power-Law Transformations- Applications

Gamma correction

- Cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with exponents varying from approximately 1.8 to 2.5.
- Gamma correction consists of using the transformation

$$s = r^{1/2.5} = r^{0.4}$$

$$s = cr^\gamma$$





$$s = cr^\gamma$$

Power-Law Transformations- Applications

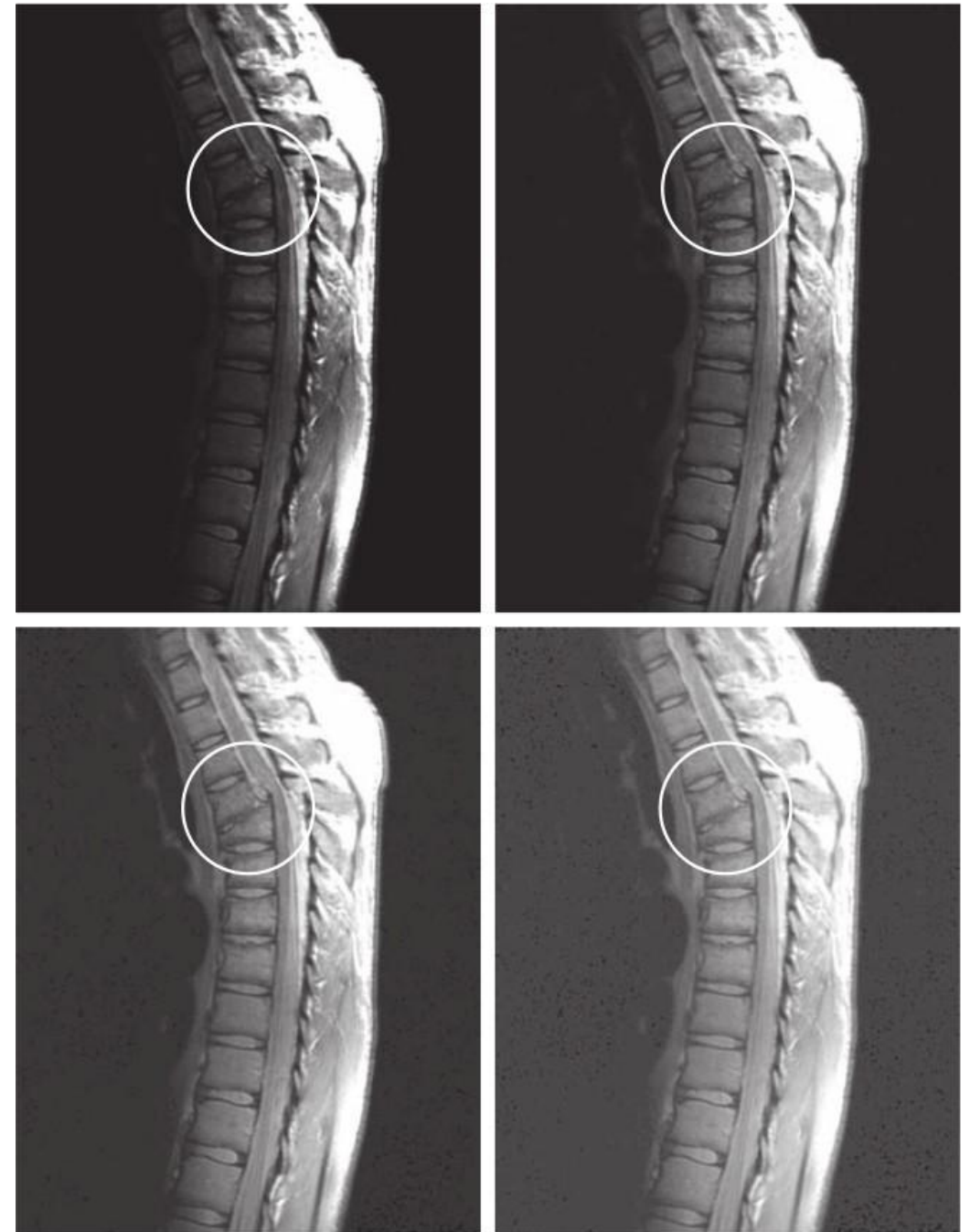
Medical Imaging

- (a) Magnetic resonance image (MRI) of a fractured human spine (the region of the fracture is enclosed by the circle).
- (b) –(d) Results of applying the transformation with $c = 1$ and $g = 0.6, 0.4$, and 0.3 , respectively.

2. Medical Imaging Application

The second slide demonstrates how these same power-law transformations are used to improve the diagnostic value of medical images, such as an MRI of a fractured human spine.

- **Original Image (a):** The fracture (enclosed in the white circle) is dark and difficult to see clearly.
- **Contrast Enhancement (b–d):** By applying the transformation $s = cr^\gamma$ with $c = 1$ and decreasing values of γ (0.6, 0.4, and 0.3), the darker regions of the image are expanded.
- **The Result:** As γ decreases, the details within the fractured area become much more visible, aiding in medical analysis.





$$s = cr^\gamma$$

Power-Law Transformations- Applications

Aerial Imaging

- (a) Aerial image.
(b)–(d) Results of applying the transformation in with $\gamma = 3.0, 4.0,$ and $5.0,$ respectively. ($c = 1$ in all cases.)

3. Aerial Imaging: Reducing Washout

Conversely, power-law transformations can also be used to darken 'washed-out' images.

- **Original Image (a):** A high-altitude aerial view appears gray and lacks contrast.
- **Transformation:** By using γ values greater than 1 (such as 3.0, 4.0, and 5.0), the image is compressed toward darker values.
- **The Result:** This increases the contrast, making features on the ground appear sharper and more distinct.



a	b
c	d



Elementary Intensity Transformations

Contrast Stretching; Thresholding

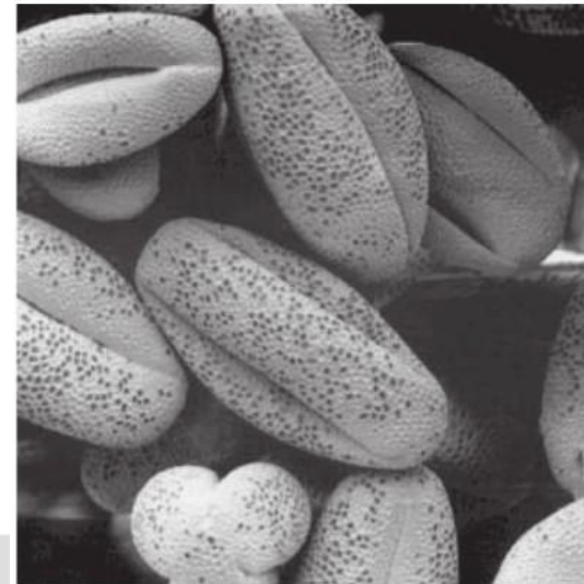
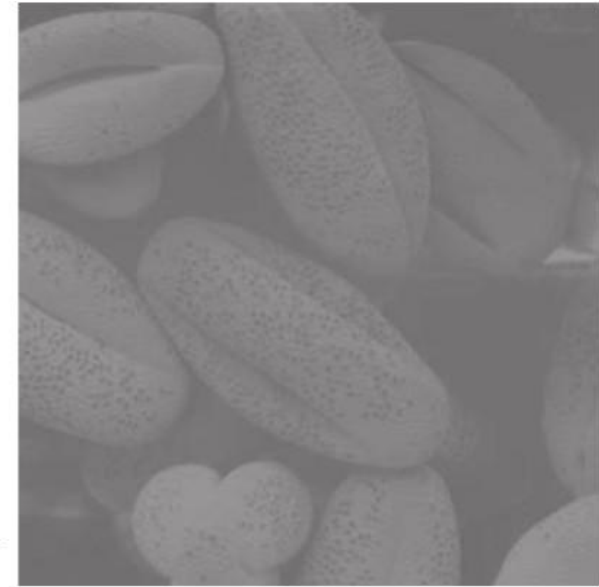
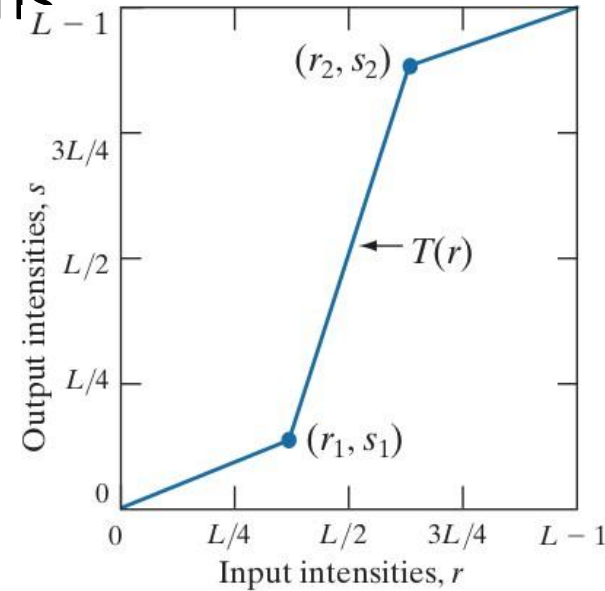
- (a) Piecewise linear transformation function.
- (b) A low-contrast electron microscope image of pollen, magnified 700 times.
- (c) Result of contrast stretching.
- (d) Result of thresholding.

Contrast Stretching and Thresholding. Both are used to manipulate the intensity (brightness) of pixels to make certain features more visible.

1. The Transformation Function (Graph a)

The graph shows a **piecewise linear transformation**, where r is the input intensity (original image) and s is the output intensity (processed image).

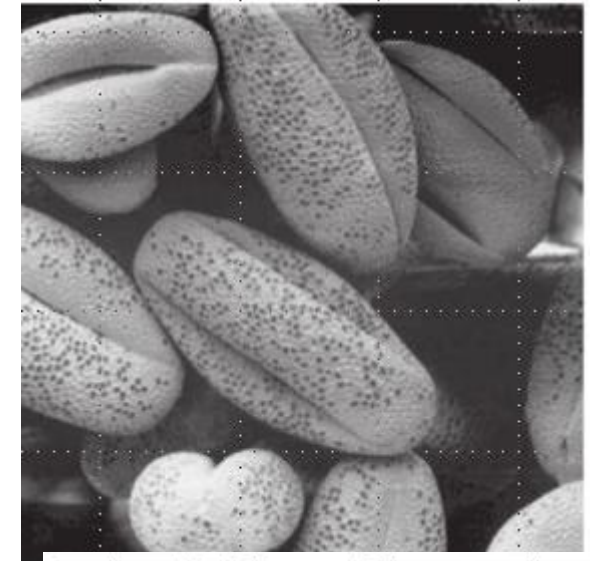
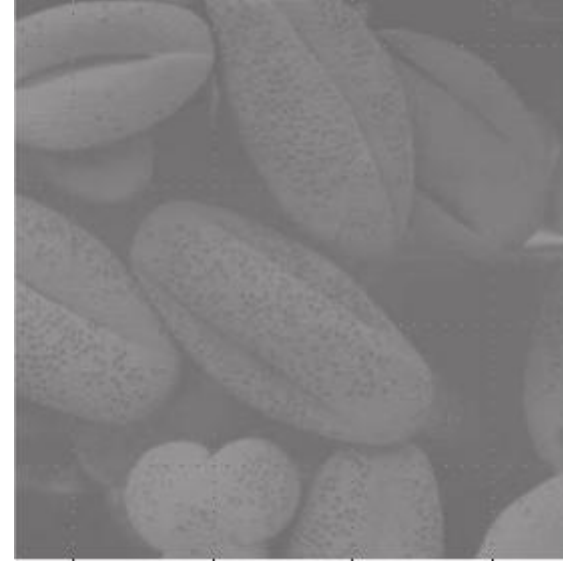
- **Linear Segments:** The function is defined by two points, (r_1, s_1) and (r_2, s_2) .
- **Contrast Stretching:** By making the slope between these two points steeper than the rest of the graph, you "stretch" a narrow range of input intensities over a wider range of output intensities. This increases the visual detail in that specific range.
- **Thresholding (The Extreme Case):** If you were to set $r_1 = r_2$, the slope would become vertical. This would turn the image into a binary (black and white) image, which is what you see in image (d).





2. The Visual Examples (Images b, c, d)

- **(b) Low-Contrast Image:** This is the original electron microscope image of pollen. Notice how it looks "washed out" or gray. Most of the pixel values are clustered in a narrow range of intensities, making it hard to see fine details.
- **(c) Contrast Stretching:** This is the result of applying the transformation function. The dark areas have become darker and the light areas lighter. The "pitted" texture on the pollen grains is now much more distinct because the dynamic range has been expanded.
- **(d) Thresholding:** This is a radical version of the transformation. Every pixel below a certain intensity threshold is turned **black**, and every pixel above it is turned **white**. This is incredibly useful for "segmenting" an image—essentially separating the objects (pollen) from the background.

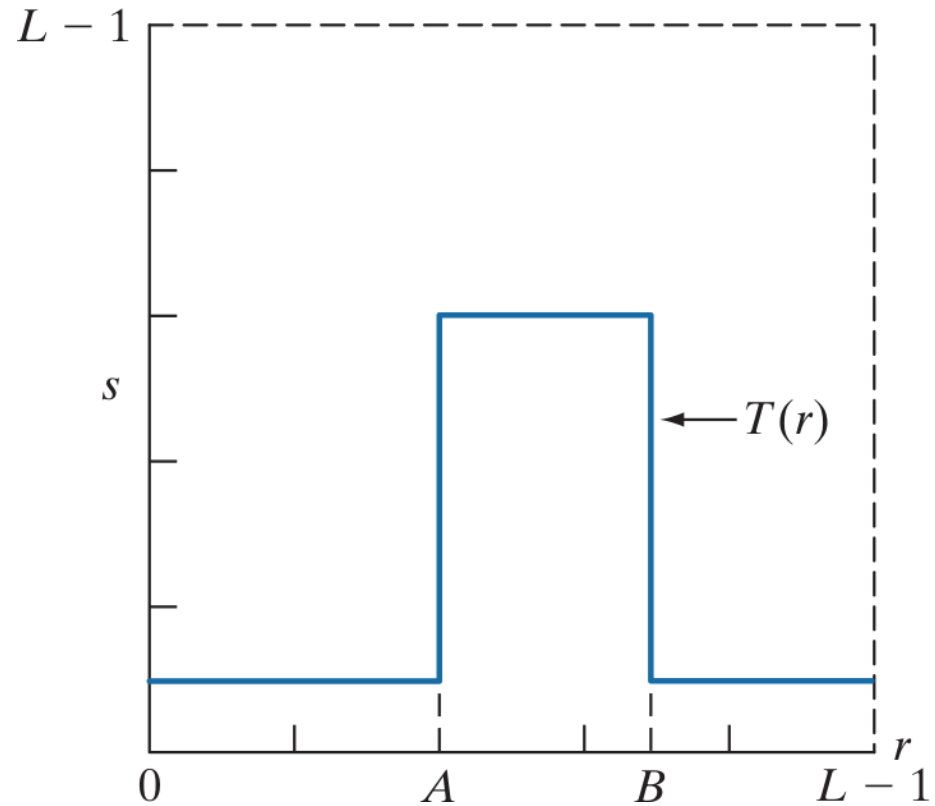


Technique	Goal	Visual Result
Contrast Stretching	Expand the range of intensity levels.	Improved detail and "pop" in colors/grays.
Thresholding	Simplify the image into two levels.	High-contrast, binary (B&W) image for shape analysis.

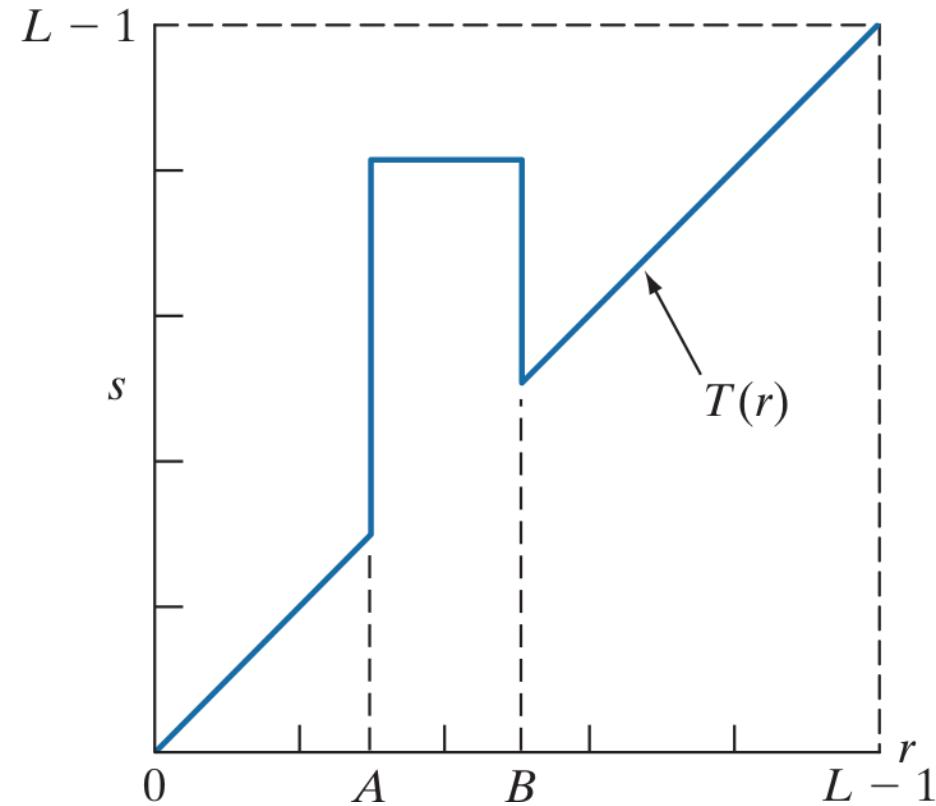


Elementary Intensity Transformations

Intensity Level Slicing



Highlights range $[A, B]$ and reduces all other intensities to a lower level.



This function highlights range $[A, B]$ and leaves other intensities unchanged.



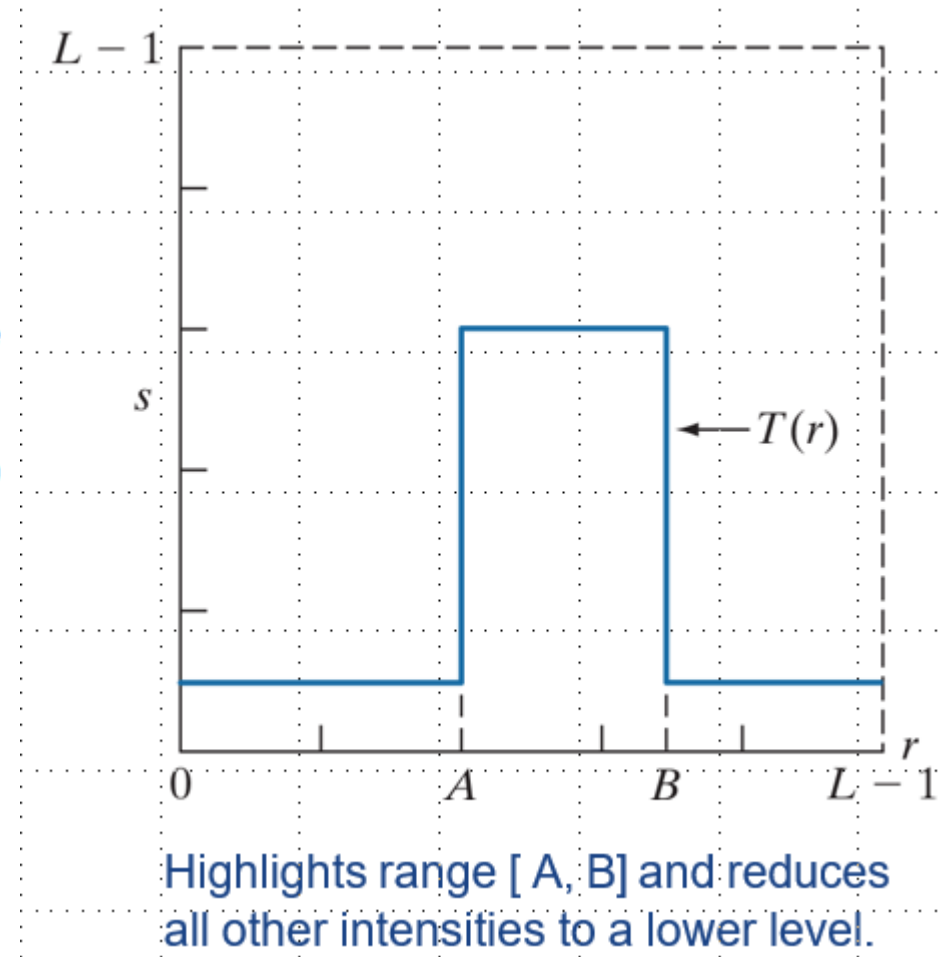
Intensity Level Slicing

Intensity Level Slicing, an image processing technique used to highlight a specific range of brightness values (intensities) within an image. This is particularly useful for making specific features—like satellite masses or water bodies—stand out from the rest of the image.

1. Highlight and Suppress (Left Graph)

This approach is more "aggressive" in how it separates features:

- **The Targeted Range:** Pixels with intensities between values $[A, B]$ are mapped to a high brightness level.
- **The Background:** All other intensities (those outside the range $[A, B]$) are reduced to a single, constant lower level.
- **Result:** This creates a binary-like effect where the features of interest are bright and everything else is "pushed" into the background.



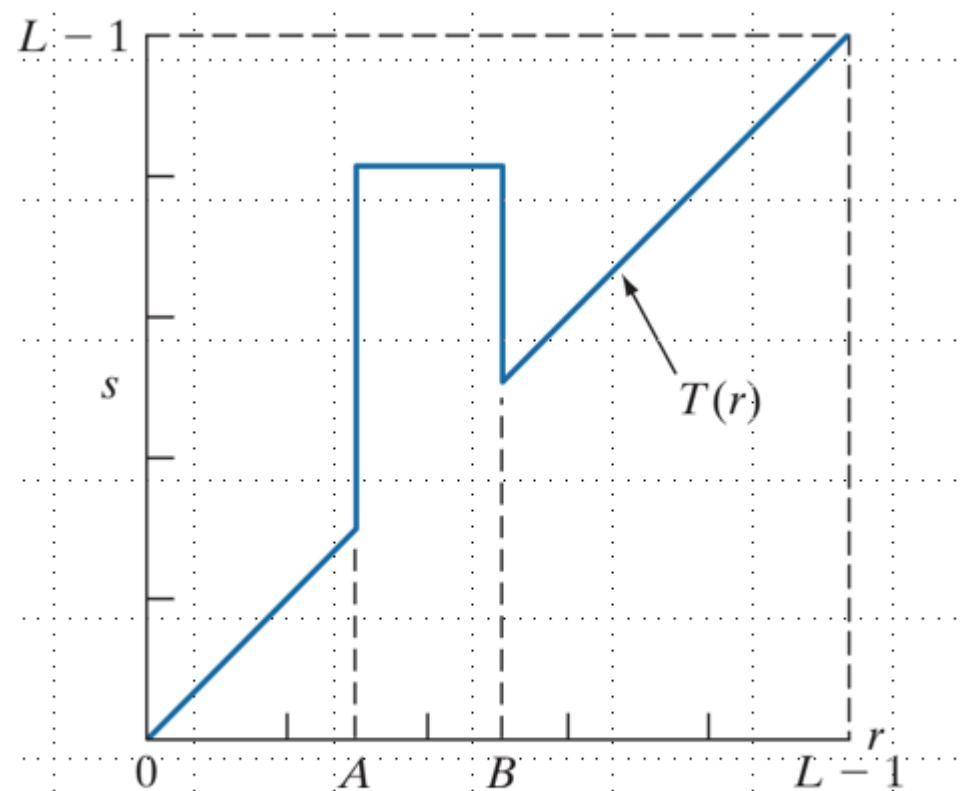


Intensity Level Slicing

2. Highlight and Preserve (Right Graph)

This approach is more subtle and preserves the context of the original image:

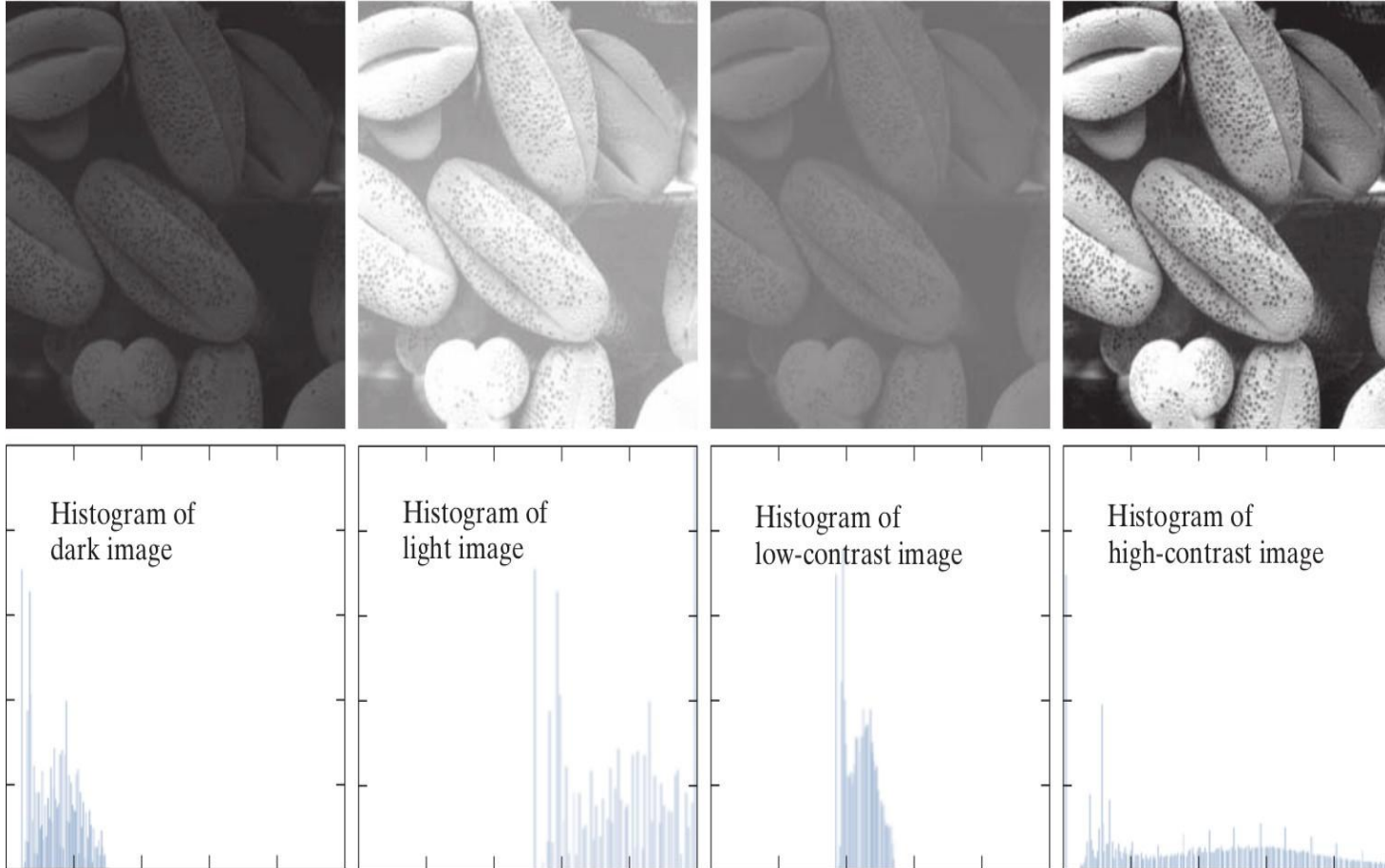
- **The Targeted Range:** Just like the first method, pixels between $[A, B]$ are boosted to a higher intensity level.
- **The Background:** Unlike the first method, all other intensities are **left unchanged**. Notice how the line follows a 45-degree angle ($s = r$) outside of the $[A, B]$ range.
- **Result:** The specific features are highlighted, but the rest of the image maintains its original tonal detail and background information.



This function highlights range $[A, B]$ and leaves other intensities unchanged.

Feature	Highlight & Suppress (Left)	Highlight & Preserve (Right)
Output for range $[A, B]$	Constant High Level	Constant High Level
Output for other values	Constant Low Level	Original Input Intensity ($s = r$)
Visual Goal	Total isolation of a feature	Highlighting a feature within its original context

Image Histogram



2. Image Histograms

A histogram is a graphical representation showing the distribution of pixel intensities in an image. The shape of the histogram directly relates to the image's appearance:

- **Dark Image:** The histogram is concentrated on the **left side** (low intensity values).
- **Light Image:** The histogram is concentrated on the **right side** (high intensity values).
- **Low-Contrast Image:** The histogram is very narrow and "bunched up" in the **middle**. This results in a "flat" or gray look because there isn't much difference between the darkest and lightest pixels.
- **High-Contrast Image:** The histogram is **spread out** across the entire range. This results in an image with deep blacks, bright whites, and distinct details.

Histogram shape is related to image appearance ;



Histogram Equalization

Histogram Equalization, a popular image processing technique used to improve contrast by spreading out the most frequent intensity values.

1. Defining the Histogram

The slide defines the normalized histogram (probability density function) of an image:

$$p(r) = \left(\frac{\text{Number of pixels with intensity } r}{\text{Total number of pixels}} \right)$$

- r : Represents the intensity levels of the original image, ranging from 0 to $L - 1$.
- $p(r)$: Represents the probability of occurrence of intensity level r .

2. The Transformation Function (T)

To change the appearance of an image, we apply a mapping function, T , to the original pixels:

$$s = T(r)$$

- r : The original pixel value.
- s : The new pixel value after processing.

Required Assumptions for $T(r)$

For the image to remain "sensible" after processing, the slide notes two critical mathematical constraints:

1. **Monotonically Increasing:** As r increases, s must also increase (or stay the same). This ensures that bright pixels don't suddenly become darker than pixels that were originally darker than them.
2. **Range [0, 1]:** Both the input and output intensities are normalized between 0 and 1. This ensures the output stays within the valid display range (e.g., not exceeding the maximum brightness).



Histogram Equalization

Histogram :

$$p(r) = \left(\frac{\text{Number of pixels with intensity } r}{\text{Total number of pixels}} \right)$$

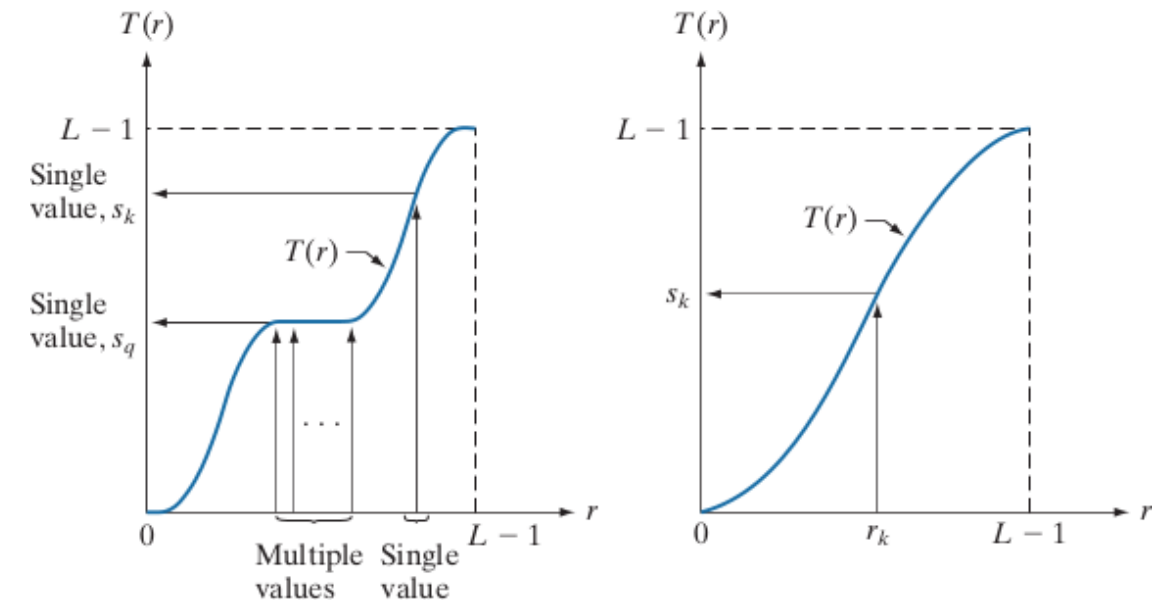
To do :

Intensity mapping

$$s = T(r)$$

Assume

- $T(r)$ is single-valued and monotonically increasing.
- $0 \leq T(r) \leq 1$ and $0 \leq r \leq 1$



- **Left Graph (Non-Strictly Monotonic):** This shows a function that "flattens" in some areas. Notice the "Multiple values" arrows—this means different input intensities (r) are being mapped to the exact same output intensity (s_q). This results in a loss of detail because distinct shades are merged into one.
- **Right Graph (Strictly Monotonic):** This shows a one-to-one mapping. Every unique input r_k results in a unique output s_k . This preserves the original order and distinctions between shades.

[†] A function $T(r)$ is a *monotonic increasing* function if $T(r_2) \geq T(r_1)$ for $r_2 > r_1$. $T(r)$ is a *strictly monotonic increasing* function if $T(r_2) > T(r_1)$ for $r_2 > r_1$. Similar definitions apply to a monotonic decreasing function.



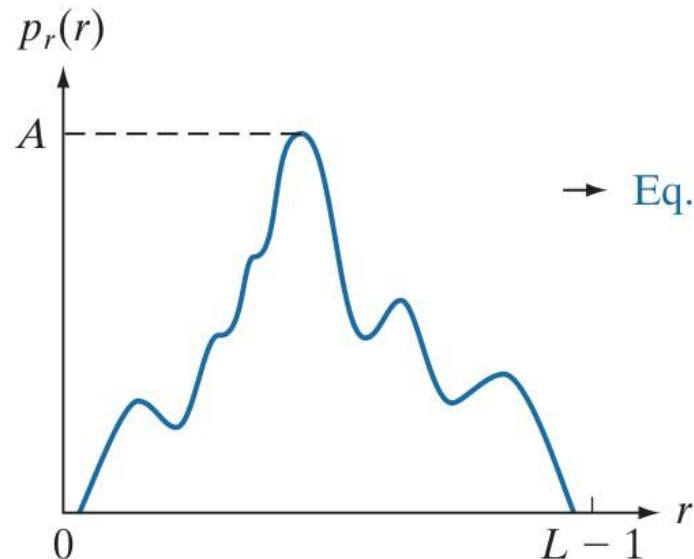
Ultimate goal of Histogram Equalization: transforming an image so that its intensity levels are distributed evenly across the entire available range.

Histogram :

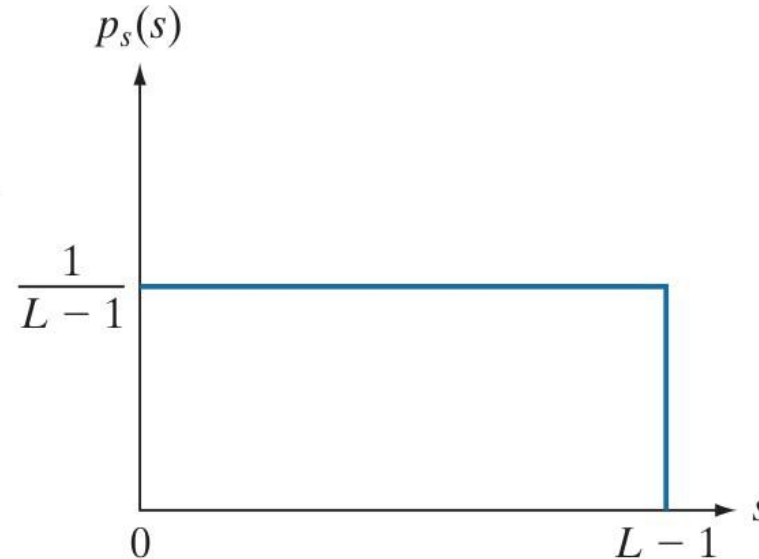
$$p(r) = \left(\frac{\text{Number of pixels with intensity } r}{\text{Total number of pixels}} \right)$$

The two graphs show the "Before" and "After" states of the process:

- **Left Graph ($p_r(r)$):** This shows a typical **original histogram** where pixel intensities are "clumped" together in certain areas. This often results in low contrast because many parts of the image share similar brightness levels.
- **Right Graph ($p_s(s)$):** This represents the **ideal output** after equalization. The histogram is now **flat (uniform)**, meaning every intensity level from 0 to $L - 1$ has an equal probability of occurring ($1/(L - 1)$).



→ Eq. (3-11) →





Histogram Equalization

$$s = T(r) \quad 0 \leq r \leq L-1$$

a. $T(r)$ is a strictly monotonically increasing function

in the interval $0 \leq r \leq L-1$;

b. $0 \leq T(r) \leq L-1$ for $0 \leq r \leq L-1$.

If $p_r(r)$ are known, and

$T(r)$ is continuous and differentiable over the range of values of interest, $r(r)$ and $T(r)$,

then the PDF of the transformed

(mapped) variable s can be obtained as

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

1. The Transformation Requirements

To perform histogram equalization, we use a transformation function $s = T(r)$ that maps an input intensity r to a new intensity s . For the transformation to work correctly, it must meet two conditions:

- **Strict Monotonicity:** $T(r)$ must be a strictly monotonically increasing function for $0 \leq r \leq L-1$. This ensures that the order of intensities is preserved (light pixels don't become darker than dark pixels) and that the mapping is one-to-one.
- **Range Constraints:** The output values $T(r)$ must stay within the range $[0, L-1]$, ensuring the new intensities are valid for the display.

2. Finding the New Probability Density Function (PDF)

If we know the original histogram $p_r(r)$ and the transformation function $T(r)$, we can calculate the PDF of the transformed image, $p_s(s)$, using the following formula:

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

- $p_s(s)$: The PDF of the new, transformed image intensities.
- $p_r(r)$: The PDF of the original image intensities.
- $|dr/ds|$: The absolute value of the derivative of r with respect to s . This represents how the transformation "stretches" or "compresses" different parts of the intensity range.



Histogram Equalization

If $p_r(r)$ and $T(r)$ are known, and

$T(r)$ is continuous and differentiable over the range of values of interest,
then the PDF of the transformed (mapped) variable s can be obtained as

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

A transformation function of particular importance in image processing is

$$s = T(r) = (L - 1) \int_0^r p_r(w) dw$$



Histogram Equalization

1. The Core Probability Theory

The first part of the slide establishes the relationship between the original pixel intensities (r) and the new, transformed intensities (s).

If we know the **Probability Density Function (PDF)** of the input, $p_r(r)$, and we apply a transformation function $s = T(r)$, the PDF of the output, $p_s(s)$, is calculated using:

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

- **What this means:** The output distribution depends on the original distribution and the "rate of change" (the derivative) of the transformation function.

Why do we do this?

The ultimate mathematical goal is to make $p_s(s)$ a **uniform distribution**. When the PDF is uniform, the image uses every possible gray level equally, resulting in the highest possible global contrast.

Fun Fact: While the math here uses continuous integrals, in actual digital image processing, we use **summations** because pixel values are discrete (0, 1, 2... 255).

2. The Histogram Equalization Formula

The second part introduces the specific transformation function used to achieve equalization. It uses the **Cumulative Distribution Function (CDF)**:

$$s = T(r) = (L - 1) \int_0^r p_r(w) dw$$

Breakdown of the variables:

- $L - 1$: This represents the maximum intensity value (e.g., **255** for an 8-bit image).
- **The Integral:** This is the CDF, which calculates the cumulative probability of all pixel values up to r .
- **The Logic:** By using the CDF as the transformation, we "stretch" the areas of the histogram that have many pixels and "compress" the areas with very few.



Histogram Equalization

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

trying to compute

$$\frac{ds}{dr} = \frac{dT(r)}{dr}$$

$$= (L-1) \frac{d}{dr} \left[\int_0^r p_r(w) dw \right]$$

$$= (L-1) p_r(r) \quad \text{Leibniz's rule in calculus}$$

$$s = T(r) = (L-1) \int_0^r p_r(w) dw$$

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

$$= p_r(r) \left| \frac{1}{(L-1)p_r(r)} \right|$$

$$= \frac{1}{L-1} \quad 0 \leq s \leq L-1$$

Form of uniform probability density function



Mathematical proof: Cumulative **Distribution Function (CDF)** -ideal transformation for histogram equalization.

Step 1: Differentiating the Transformation Function

To find the relationship between the input and output, we first differentiate the transformation function $s = T(r)$ with respect to r :

$$\frac{ds}{dr} = \frac{dT(r)}{dr} = (L - 1) \frac{d}{dr} \left[\int_0^r p_r(w) dw \right]$$

Using **Leibniz's rule** (the Fundamental Theorem of Calculus), the derivative of the integral of the PDF is simply the PDF itself:

$$\frac{ds}{dr} = (L - 1)p_r(r)$$

Step 2: Substituting into the PDF Formula

The slide then takes the general formula for the PDF of a transformed variable, $p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$, and substitutes the result from Step 1.

Note that since $\frac{ds}{dr} = (L - 1)p_r(r)$, its reciprocal is:

$$\left| \frac{dr}{ds} \right| = \frac{1}{(L - 1)p_r(r)}$$

Plugging this into the main equation:

$$p_s(s) = p_r(r) \cdot \frac{1}{(L - 1)p_r(r)}$$

Step 3: The Final Result

After the $p_r(r)$ terms cancel out, we are left with a constant:

$$p_s(s) = \frac{1}{L - 1} \quad \text{for } 0 \leq s \leq L - 1$$

Key Takeaway

The result $\frac{1}{L-1}$ is the **form of a uniform probability density function**. This proves that if you map pixel intensities using their own cumulative distribution, the resulting image will have an **equalized histogram** where every intensity level is equally likely to occur.



Histogram Equalization

$T(r)$ for continuous values:

$$s = T(r) = (L - 1) \int_0^r p_r(w) dw$$

Need to just use this to transform image

Digital image is discrete:

$$p_r(r_k) = \frac{n_k}{MN} \quad s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) \quad k = 0, 1, 2, \dots, L - 1$$

From Continuous to Discrete

- **Continuous:** Uses integrals ($\int$) to represent the area under a curve.
- **Discrete:** Digital images use summations ($\sum$) because pixel values are distinct integers (like 0, 1, 2... 255).
- **Probability of a Gray Level:** $p_r(r_k) = \frac{n_k}{MN}$.
 - n_k is the number of pixels with intensity k , and MN is the total number of pixels in the image.
- **The Transformation Function:** $s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j)$.
 - This is the "working" formula. You calculate the running sum of probabilities up to level k , multiply by the max intensity ($L - 1$), and the result is your new pixel value.



Histogram Equalization - Example

Suppose that a 3-bit image ($L=8$) of size 64×64 pixels ($MN = 4096$) has the intensity distribution shown in following table.

r_k	n_k	$p_r(r_k) = n_k / MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02



Histogram Equalization - Example

r_k	n_k	$p_r(r_k) = n_k / MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

$$s_0 = T(r_0) = 7 \sum_{j=0}^0 p_r(r_j) = 7 \times 0.19 = 1.33 \rightarrow 1$$

$$s_1 = T(r_1) = 7 \sum_{j=0}^1 p_r(r_j) = 7 \times (0.19 + 0.25) = 3.08 \rightarrow 3$$

$$s_2 = 4.55 \rightarrow 5 \quad s_3 = 5.67 \rightarrow 6$$

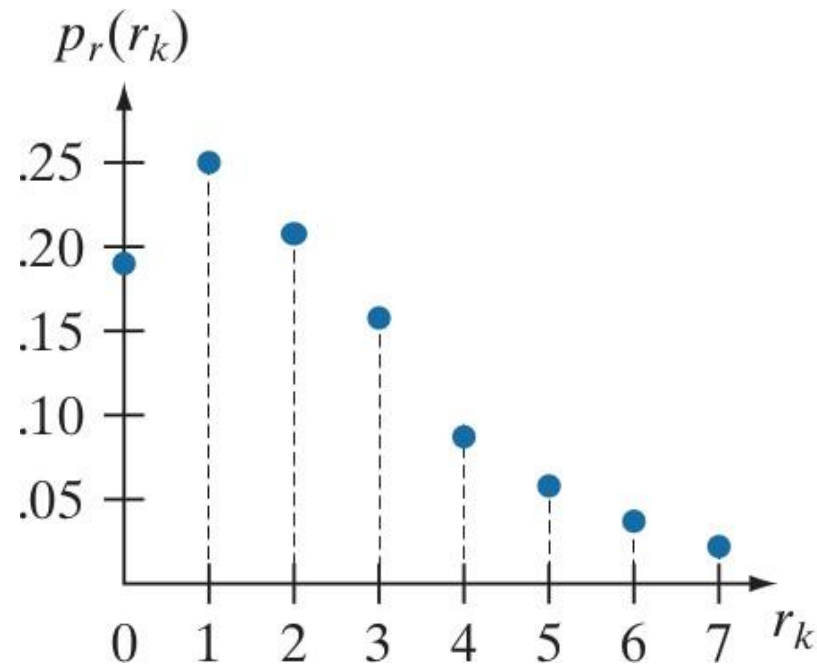
$$s_4 = 6.23 \rightarrow 6 \quad s_5 = 6.65 \rightarrow 7$$

$$s_6 = 6.86 \rightarrow 7 \quad s_7 = 7.00 \rightarrow 7$$

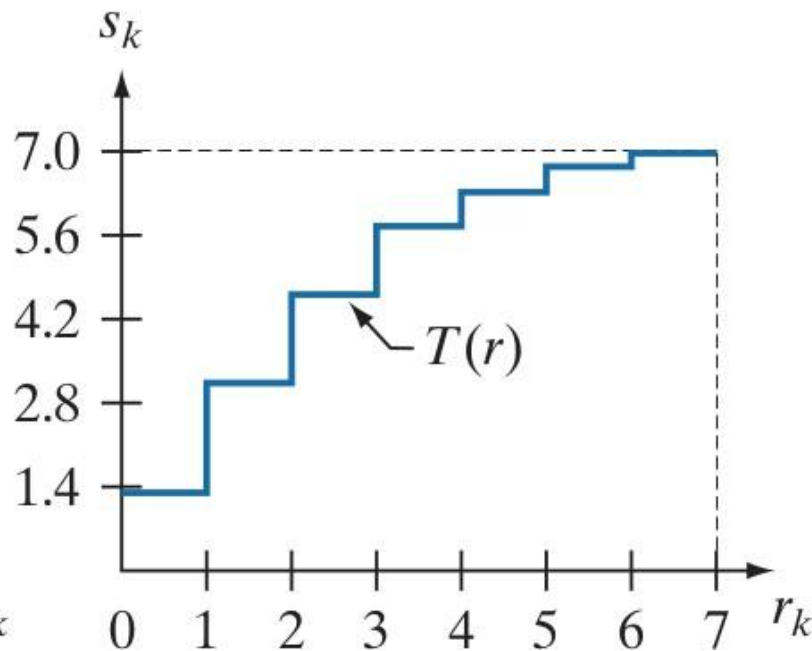


Histogram Equalization - Example

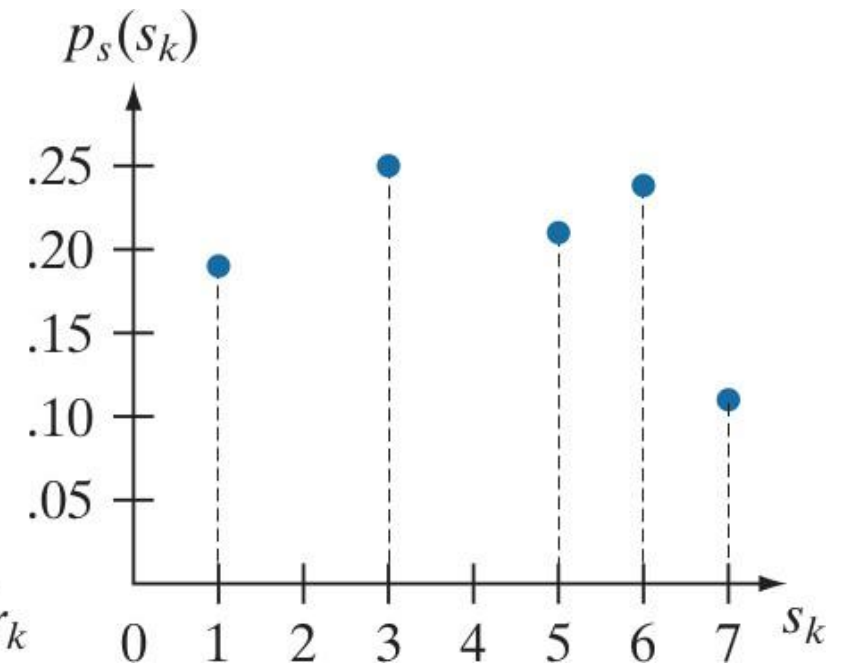
$$\begin{array}{llll} s_0 = 1.33 \rightarrow 1 & s_2 = 4.55 \rightarrow 5 & s_4 = 6.23 \rightarrow 6 & s_6 = 6.86 \rightarrow 7 \\ s_1 = 3.08 \rightarrow 3 & s_3 = 5.67 \rightarrow 6 & s_5 = 6.65 \rightarrow 7 & s_7 = 7.00 \rightarrow 7 \end{array}$$



Original histogram.



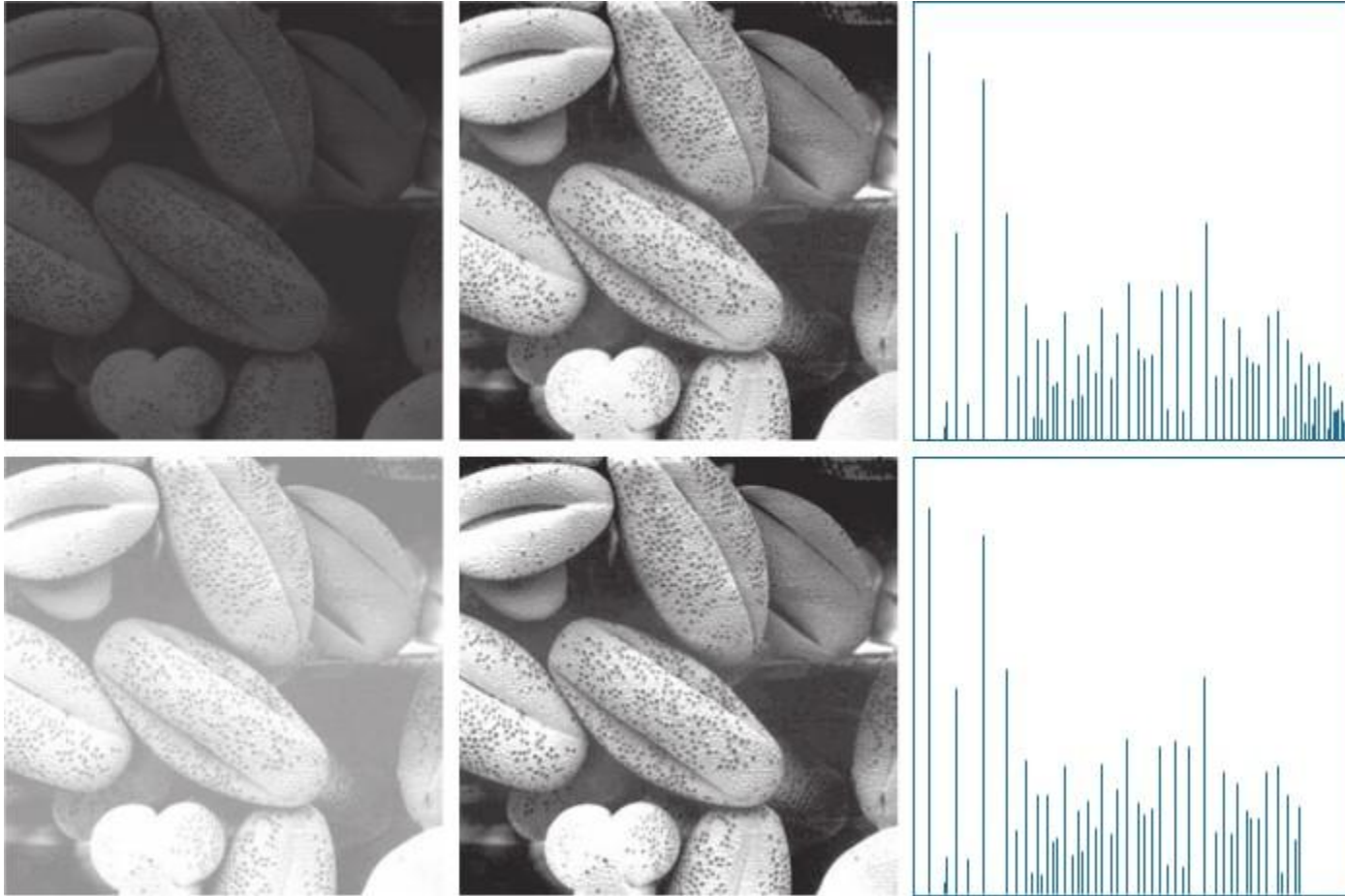
Transformation function.



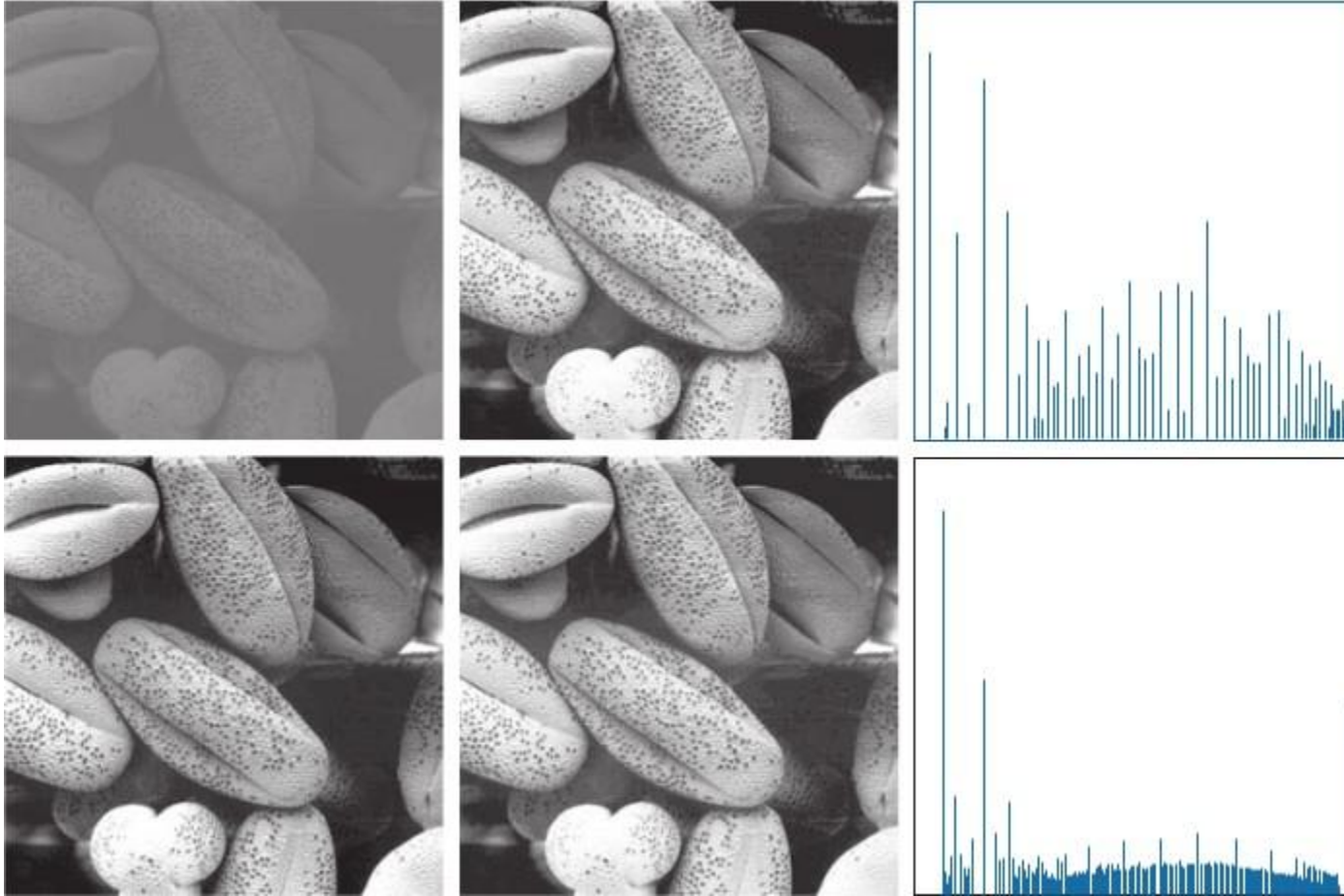
Equalized histogram.



Histogram Equalization - Example



Histogram Equalization - Example





Spatial Filtering

Spatial Filtering (often called convolution or correlation), which is a cornerstone of digital image processing used for tasks like blurring, sharpening, and edge detection.

The core idea is moving a small matrix (the **kernel**) over an image to calculate new pixel values based on their neighbors.

1. The Core Components

To perform spatial filtering, you need two things:

- **The Input Image:** A grid of pixels, where each pixel has a specific intensity.
- **The Filter Kernel (or Mask):** A small matrix (usually 3×3 , 5×5 , or 7×7) that contains coefficients or "weights."

2. How it Works (The "Slide and Sum" Method)

The process is an iterative operation often referred to as convolution or correlation:

- **Alignment:** Place the center of the kernel over a specific pixel (x, y) in the image.
- **Multiplication:** Multiply each coefficient in the kernel by the corresponding pixel value it is currently sitting over.
- **Summation:** Add all those multiplication results together.
- **Replacement:** The final sum becomes the new value for the pixel at (x, y) in the output image.
- **Iteration:** Move (slide) the kernel to the next pixel and repeat until the entire image is processed.

4. Mathematical Representation

If the kernel is w and the image is f , the response $g(x, y)$ at any point is:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

- a and b represent the distance from the center to the edge of the kernel.
- For a 3×3 kernel, $a = 1$ and $b = 1$.



Spatial Filtering

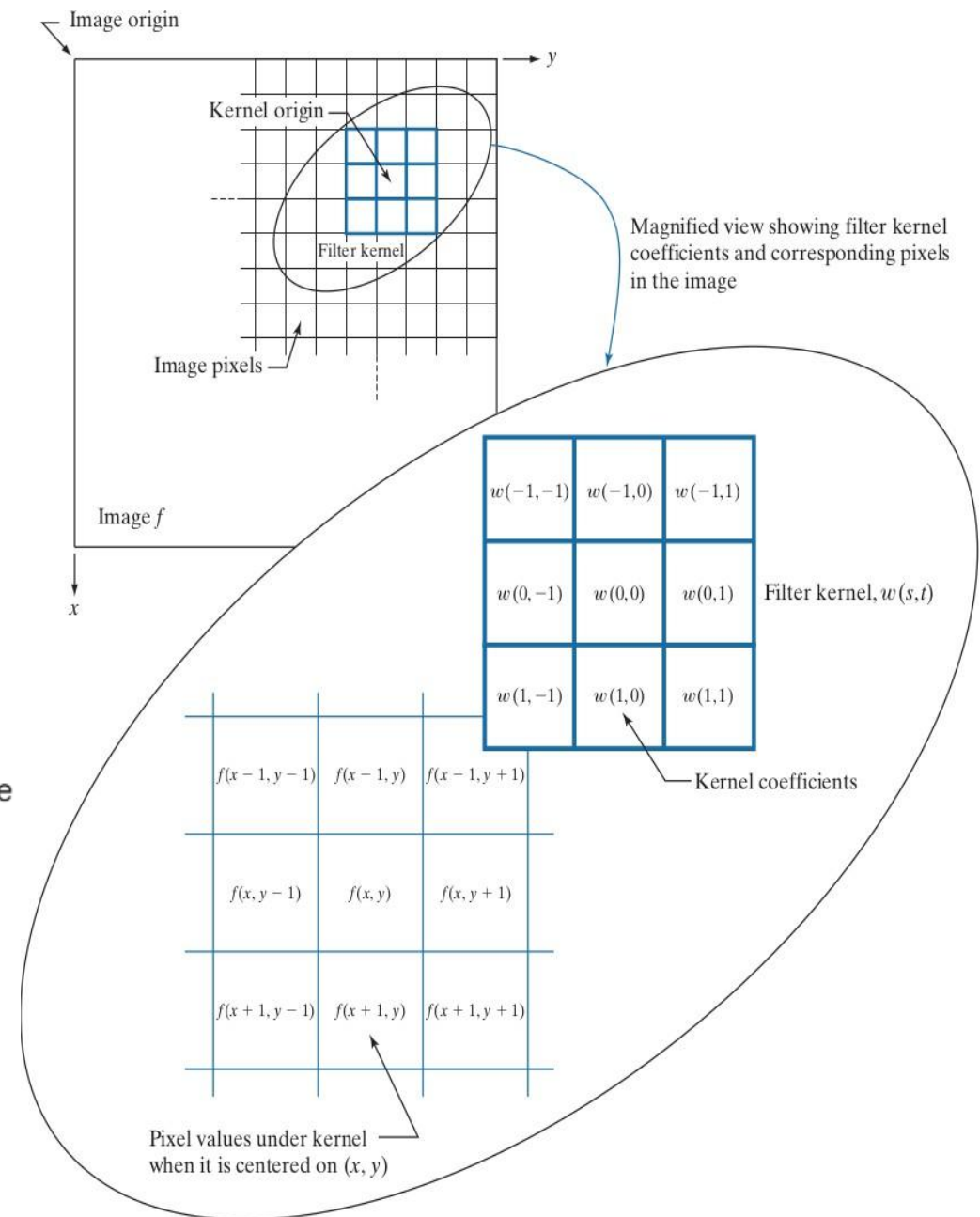
Note:

Origin of the image is at the top left, but the origin of the kernel is at its center. Placing the origin at the center of spatially symmetric kernels simplifies writing expressions for linear filtering.

1. The Image vs. The Kernel

The slide makes a critical distinction between how we index the image and how we index the filter:

- **Image Origin:** Located at the **top-left corner** $(0, 0)$. Any pixel in the image is denoted as $f(x, y)$.
- **Kernel Origin:** Located at its **center**. For a 3×3 kernel, the center is $(0, 0)$, and the indices range from -1 to $+1$.
 - *Why?* As the note mentions, centering the origin simplifies the math. It allows us to use symmetric limits (e.g., from $-a$ to $+a$) when writing the formula for filtering.
- **Alignment:** The center of the kernel, $w(0, 0)$, is placed directly on top of the target pixel $f(x, y)$.
- **Neighborhood:** The surrounding kernel coefficients ($w(-1, -1)$ through $w(1, 1)$) align with the surrounding image pixels ($f(x - 1, y - 1)$ through $f(x + 1, y + 1)$).





Spatial Filtering

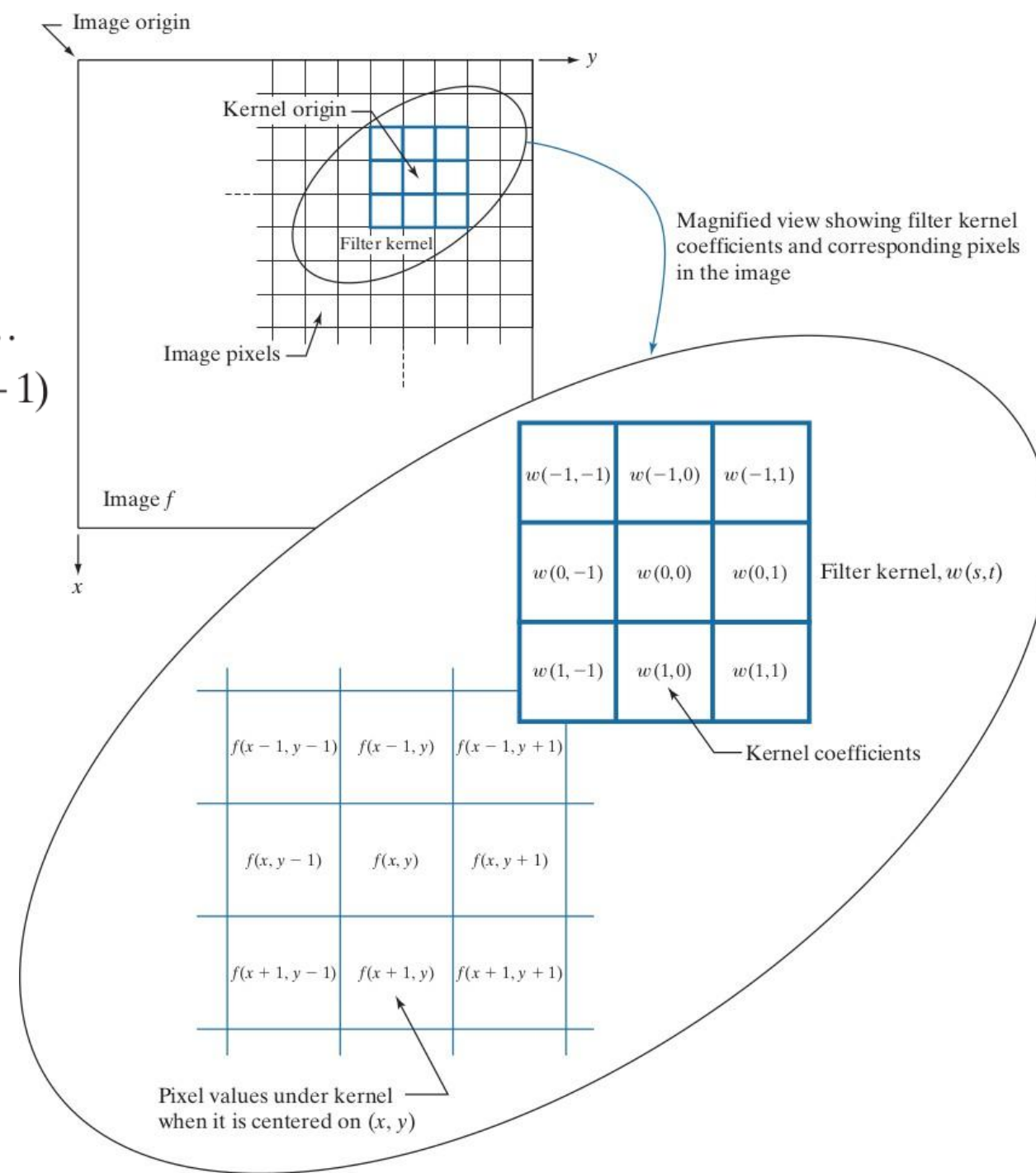
$$g(x, y) = w(-1, -1)f(x - 1, y - 1) + w(-1, 0)f(x - 1, y) + \dots \\ + w(0, 0)f(x, y) + \dots + w(1, 1)f(x + 1, y + 1)$$

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t)f(x + s, y + t)$$

Note:

$$m = 2a + 1$$

$$n = 2b + 1$$





Spatial Filtering

- A spatial filter consists of (a) **a neighborhood**, and (b) **a predefined operation**
- Linear spatial filtering of an image of size $M \times N$ with a filter of size $m \times n$ is given by the expression

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

linear spatial filtering

A spatial filter is defined by two primary elements:

- **A Neighborhood:** A small area of pixels surrounding a specific coordinate (x, y) .
- **A Predefined Operation:** A mathematical calculation performed on the pixels within that neighborhood to produce a single output value.
- **Odd-Sized Kernels:** Filters are typically odd-sized to ensure they have a clear center pixel (the origin).
- **Calculating Limits:** The total width (m) and height (n) of the filter are defined as:
 - $m = 2a + 1$
 - $n = 2b + 1$
- For a standard 3×3 filter, $a = 1$ and $b = 1$. This means the summation indices s and t run from -1 to $+1$, covering all nine pixels in the neighborhood.

The process of linear spatial filtering for an image of size $M \times N$ using a filter (kernel) of size $m \times n$ is expressed by the following summation:

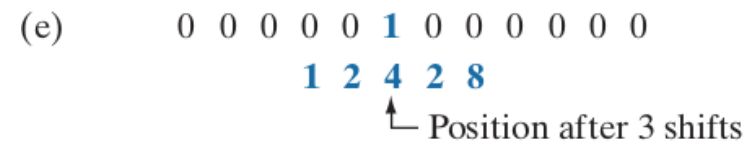
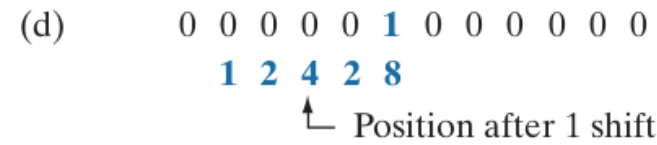
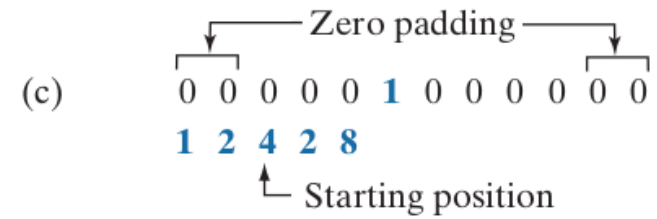
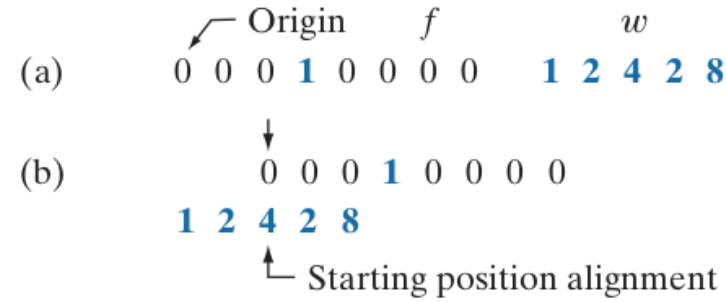
$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

- $f(x + s, y + t)$: These are the original pixel values within the neighborhood centered at (x, y) .
- $w(s, t)$: These are the **filter coefficients** or weights within the kernel.
- $g(x, y)$: This is the resulting filtered pixel value at the same coordinate in the output image.



Spatial Filtering

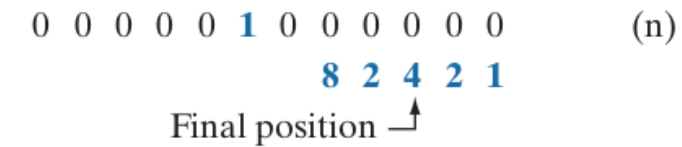
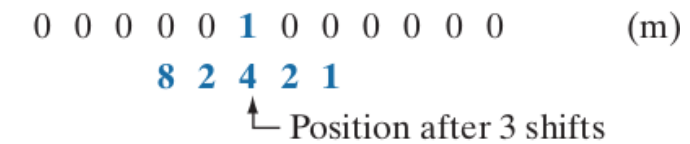
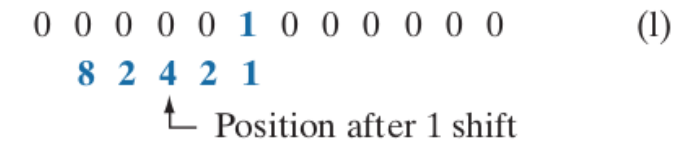
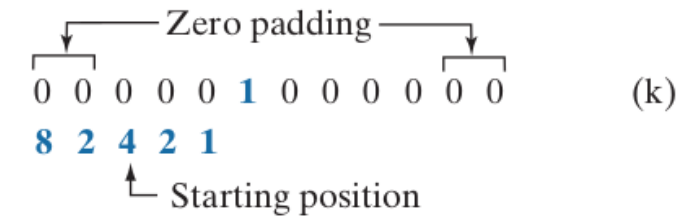
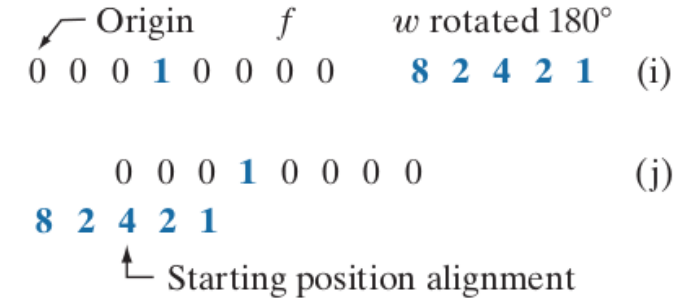
Correlation



Correlation result

(g) 0 8 2 4 2 1 0 0

Convolution



Convolution result

(o) 0 1 2 4 2 8 0 0



Spatial Filtering

		Padded f								
$\nwarrow$	Origin	f								
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	

(a)

		Padded f								
$\nwarrow$	Origin	f								
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	

(b)

↖	Initial position for w							Correlation result				
1	2	3	0	0	0	0	0	0	0	0	0	0
4	5	6	0	0	0	0	0	0	9	8	7	0
7	8	9	0	0	0	0	0	0	6	5	4	0
0	0	0	1	0	0	0	0	0	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0

(c)

↖	Rotated w							Convolution result				
9	8	7	0	0	0	0	0	0	0	0	0	0
6	5	4	0	0	0	0	0	0	1	2	3	0
3	2	1	0	0	0	0	0	0	4	5	6	0
0	0	0	1	0	0	0	0	0	7	8	9	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0

(f)

(g)



Smoothing Spatial Filters

2. Smoothing Spatial Filters (Image 2)

These are **2D filters** used to blur images, reduce noise, or remove small details.

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

$$\frac{1}{4.8976} \times \begin{array}{|c|c|c|} \hline 0.3679 & 0.6065 & 0.3679 \\ \hline 0.6065 & 1.0000 & 0.6065 \\ \hline 0.3679 & 0.6065 & 0.3679 \\ \hline \end{array}$$

Box kernel
Gaussian kernel.

The Box Kernel

- **Design:** This is a 3×3 grid where every coefficient is 1.
- **The Fraction ($\frac{1}{9}$):** This is the **normalization constant**. Since there are 9 pixels, you divide the sum by 9 so that the total weight equals 1. This ensures the output image isn't brighter than the original.
- **Effect:** It takes the average of all pixels in the neighborhood, creating a simple blurring effect.

The Gaussian Kernel

- **Design:** Unlike the Box kernel, the values here are not uniform. The center value is highest (1.0000), and values decrease as you move away from the center.
- **The Fraction ($\frac{1}{4.8976}$):** Again, this is the sum of all values in the grid, used for normalization.
- **Effect:** This produces a "weighted average." It mimics how light naturally blurs, resulting in a much smoother, more natural-looking blur than the Box kernel.

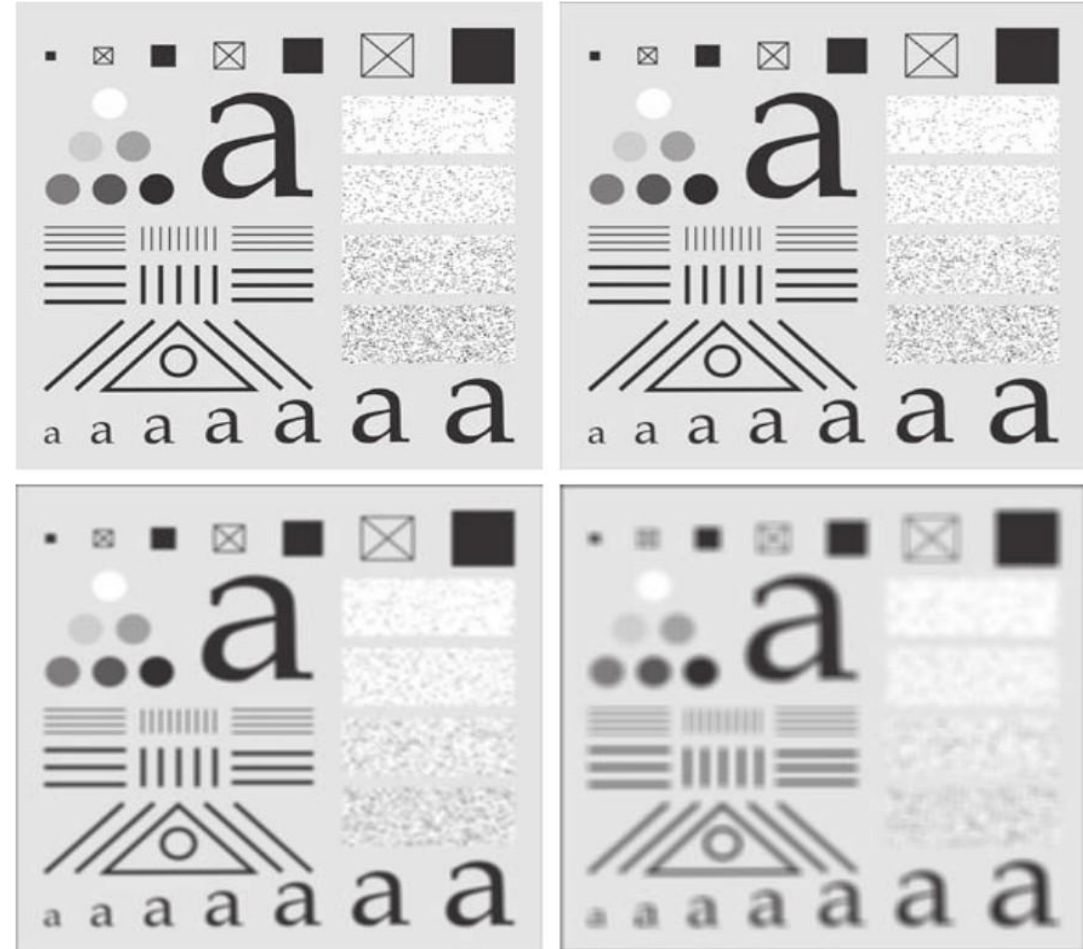


Smoothing Spatial Filters

(a) Test pattern of size 1024×1024 pixels. (b)-(d) Results of low pass filtering with **box kernels** of sizes 3×3 , 11×11 , and 21×21 , respectively.

The final slide shows how the size of a **box kernel** affects an actual test pattern image of 1024×1024 pixels:

- **Increasing Kernel Size:** The slide compares three different sizes: 3×3 , 11×11 , and 21×21 .
- **Visual Impact:**
 - 3×3 : Provides a subtle smoothing effect.
 - 11×11 and 21×21 : As the kernel size increases, the blurring becomes more intense, eventually causing smaller details (like the text and fine lines) to blend into the background.
- **Function:** These are referred to as **low pass filters** because they allow large, smooth areas to remain while removing "high-frequency" details like sharp edges and noise.

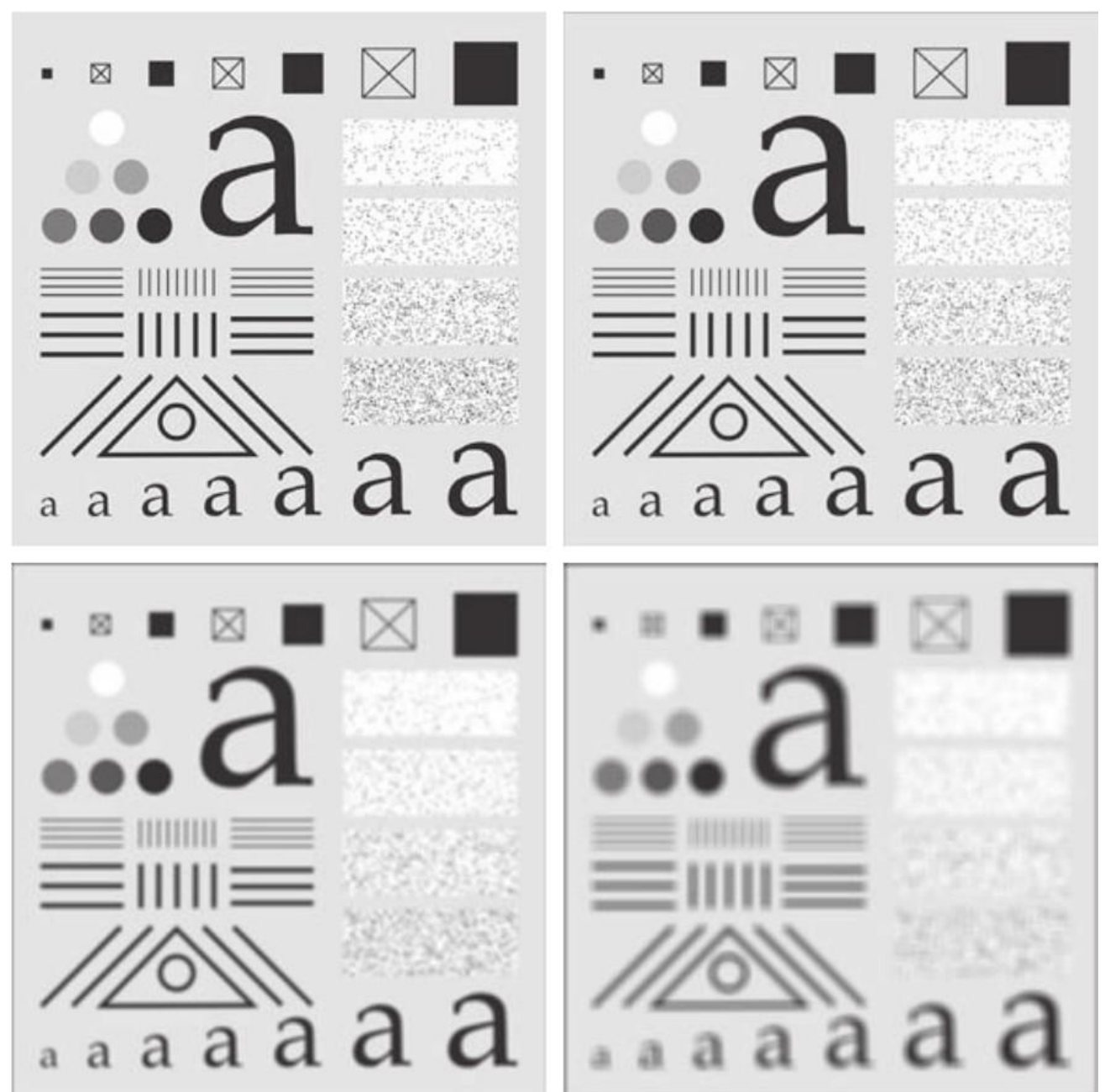




Smoothing Spatial Filters

(a) Test pattern of size 1024×1024 pixels.

(b)-(d) Results of low pass filtering
with **box kernels** of sizes 3×3 ,
 11×11 , and 21×21 , respectively.



a	b
c	d



Spatial Filtering

- A spatial filter consists of (a) **a neighborhood**, and (b) **a predefined operation**
- Linear spatial filtering of an image of size $M \times N$ with a filter of size $m \times n$ is given by the expression

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

1. What is a Spatial Filter?

A spatial filter is a tool used to modify an image based on the value of a pixel's neighbors. It consists of two main parts:

- **A Neighborhood:** A small area (usually a grid like 3×3) surrounding a pixel.
- **A Predefined Operation:** A mathematical calculation performed on the pixels within that neighborhood.

For linear spatial filtering, this operation is expressed as a double summation:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$



Gaussian Filters

Let s and r be position of a cell in a filter w.r.t the origin; r represent the distance of the cell from the center

$$r = [s^2 + t^2]^{1/2}$$

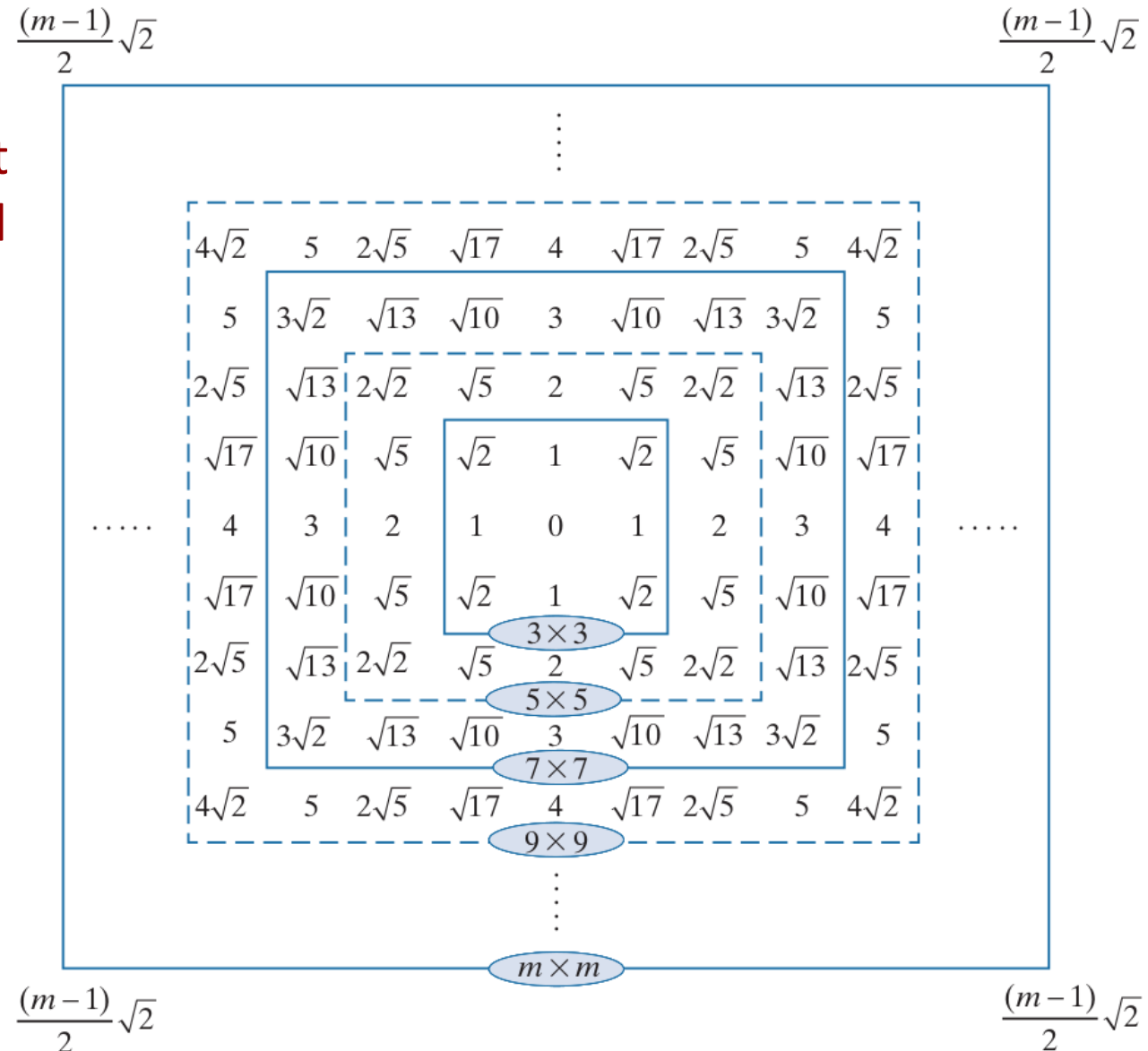
Form of Gaussian kernels is as below:

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

- **Symmetry:** Notice how the distance values are mirrored across the center. Gaussian filters are **rotationally symmetric**, meaning the blur is applied equally in all directions.

Kernel Size ($m \times m$): The slide shows that as the kernel grows, the maximum distance at the corners reaches $\frac{(m-1)}{2}\sqrt{2}$

Weight Distribution: Because of the negative exponent in the formula, cells closer to the center (r is small) get **higher weights**, while cells further away (r is large) have their influence diminished exponentially.



Distances from the center for various sizes of square kernels.



Gaussian Filters

Gaussian Filters, which are widely used in image processing for blurring and noise reduction. It focuses on how to calculate the weight of each pixel (or "cell") in a filter based on its distance from the center.

1. The Geometry: Distance (r)

In a square filter (kernel), the "center" is the origin $(0, 0)$. For any other cell at position (s, t) , its distance r from the center is calculated using the **Euclidean distance formula**:

$$r = [s^2 + t^2]^{1/2}$$

- **The Diagram:** The right side shows distance values for kernels of various sizes (3×3 , 5×5 , etc.).
 - The center cell always has a distance of **0**.
 - Immediate neighbors have a distance of **1** (horizontal/vertical) or $\sqrt{2}$ (diagonal).
 - As you move outward to larger kernels (up to $m \times m$), the distance increases.

2. The Gaussian Function $G(r)$

The value (or weight) assigned to each cell in the filter is determined by the Gaussian distribution formula:

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

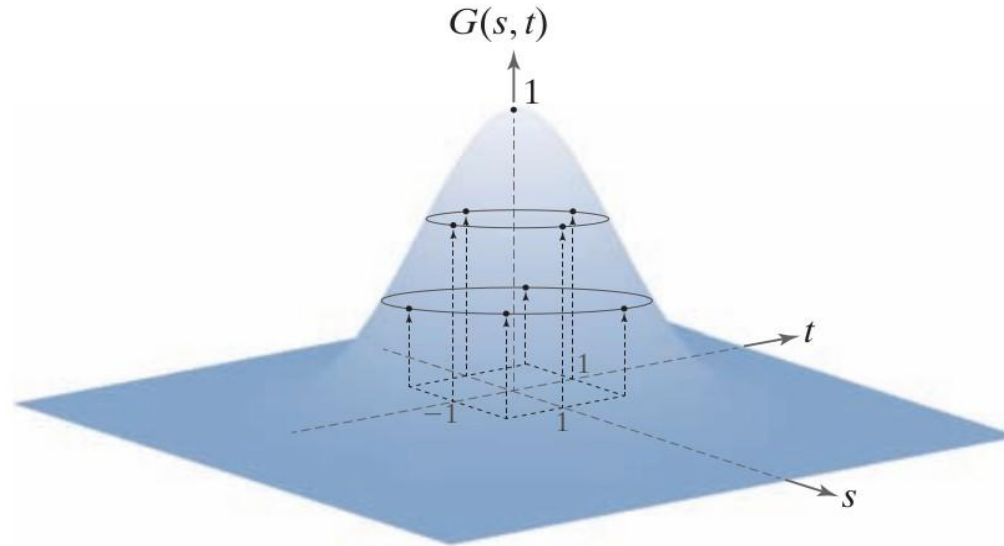
- **K (Constant):** A scaling factor (often used to ensure the sum of all weights in the filter equals 1).
- **r^2 :** The squared distance we calculated above.
- **σ (Sigma):** The standard deviation. This controls the "spread" of the blur. A larger σ results in a wider, smoother blur.



Gaussian Filters

$$r = [s^2 + t^2]^{1/2}$$

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$



$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

Resulting 3×3 kernel

Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$.

1. From 3D Surface to Discrete Points

The center image shows the **Gaussian "Bell Curve"** in three dimensions (s, t, G).

- **Sampling:** To create a digital filter, we "sample" the height of this bell curve at specific integer grid points (e.g., at coordinates $(0, 0)$, $(1, 0)$, $(1, 1)$, etc.).
- **The Peak:** The highest point is at the center $(0, 0)$, where the value is 1.0 (because $K = 1$ in this example).
- **The Slope:** As you move away from the center in any direction (s or t), the value drops, creating the weights for the surrounding pixels.

2. The Calculated Kernel Values

On the right, you see the actual resulting numbers for a 3×3 kernel where $K = 1$ and $\sigma = 1$:

- **Center:** $G(0) = 1.0000$.
- **Sides (distance $r = 1$):** Using the formula $e^{-1^2/2}$, we get 0.6065.
- **Corners (distance $r = \sqrt{2}$):** Using $e^{-(\sqrt{2})^2/2} = e^{-1}$, we get 0.3679.

Notice the fraction $\frac{1}{4.8976}$ in front of the kernel.

- **Why it's there:** If you sum up all nine values in the grid ($1.0000 + 4 \times 0.6065 + 4 \times 0.3679$), the total is 4.8976.

- **The Goal:** In image blurring, we usually want the average brightness of the image to stay the same. Dividing by this sum ensures the total weight of the filter equals 1, preventing the image from becoming brighter or darker after the filter is applied



Gaussian Filters



A test pattern of size 1024×1024



lowpass filtering the pattern with a Gaussian kernel of size 21×21 , with $\sigma = 3.5$, $K=1$



Result of using a kernel of size 43×43 , with $\sigma = 7$, $K=1$

- **Lowpass Filtering:** Gaussian filters act as "lowpass filters," meaning they allow low-frequency information (large shapes) to pass through while blocking high-frequency information (sharp edges and noise).
- **The Role of σ :** As σ increases, the "bell" of the Gaussian function becomes wider, encompassing more pixels and resulting in a much stronger blur.

Image	Kernel Size	σ (Spread)
Original	N/A	N/A
Moderate Blur	21×21	3.5
Heavy Blur	43×43	7.0



Gaussian Filters



Gaussian kernels of size 43×43 ,
with $\sigma = 7$

Kernel of 85×85 , with $\sigma = 7$

Difference image

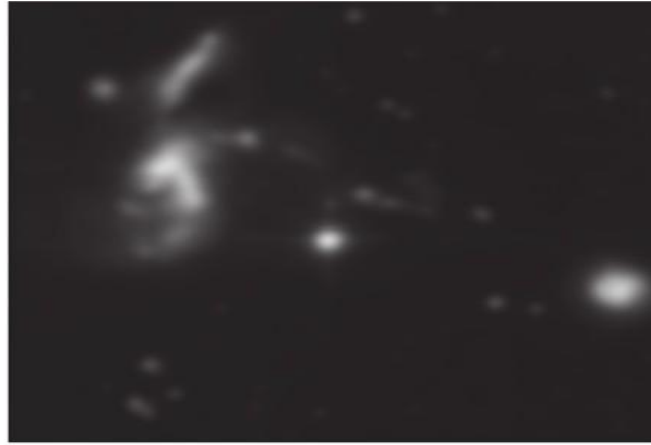
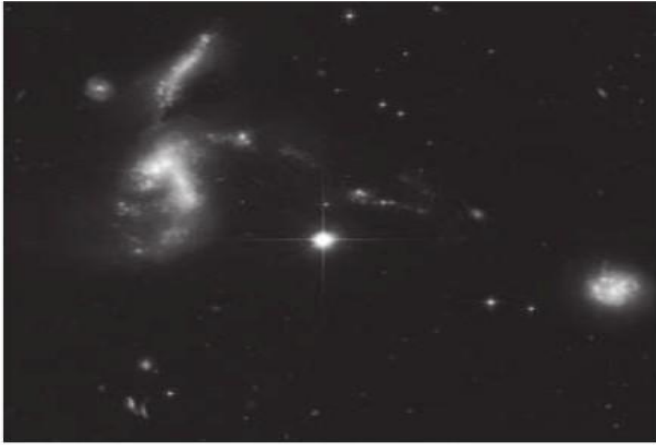
3. Understanding Kernel Truncation (Slide 53)

This slide illustrates a crucial practical concept: the relationship between the physical size of the kernel and the value of σ .

- **Comparative Test:** It compares a 43×43 kernel against a much larger 85×85 kernel, both using the same $\sigma = 7$.
- **The "Difference Image":** The black image on the right indicates that there is **zero (or negligible) difference** between the two results.
- **Key Takeaway:** Because Gaussian values drop off exponentially, once a kernel is large enough to capture the significant portion of the bell curve (typically 6σ), making it even larger provides no additional visual benefit but increases computational cost.



Gaussian Filters - Example Usage



a b c

A 2566 × 2758 Hubble Telescope image of the Hickson Compact Group

Result of lowpass filtering with a Gaussian kernel.

Result of thresholding the filtered image (intensities were scaled to the range [0, 1])

3. Example Usage: Hubble Telescope Image (Slide 54)

This slide shows a real-world application of Gaussian filtering in astronomy.

- **Original Image (a):** A raw image of the Hickson Compact Group which contains significant noise and small, distracting artifacts.
- **Filtering (b):** Applying a Gaussian lowpass filter smooths the image, reducing the noise and making the primary light sources (galaxies) more cohesive.
- **Thresholding (c):** After filtering, a threshold is applied to isolate the brightest objects. This process is much more accurate on a filtered image because the Gaussian blur removes the tiny noise spikes that would otherwise be mistakenly identified as objects.



Gaussian Filters - Example Usage

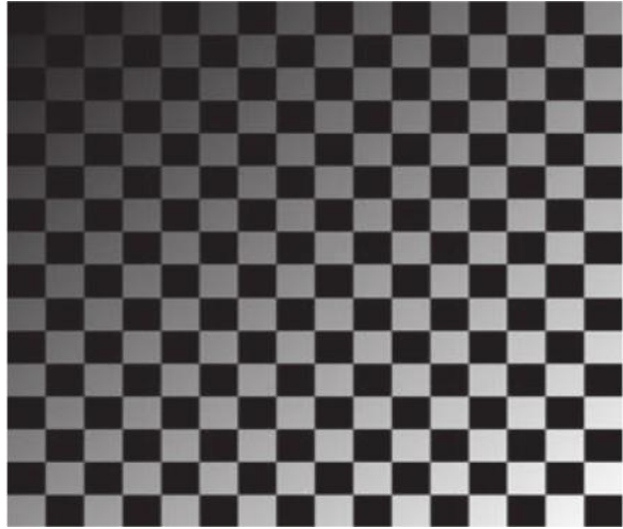
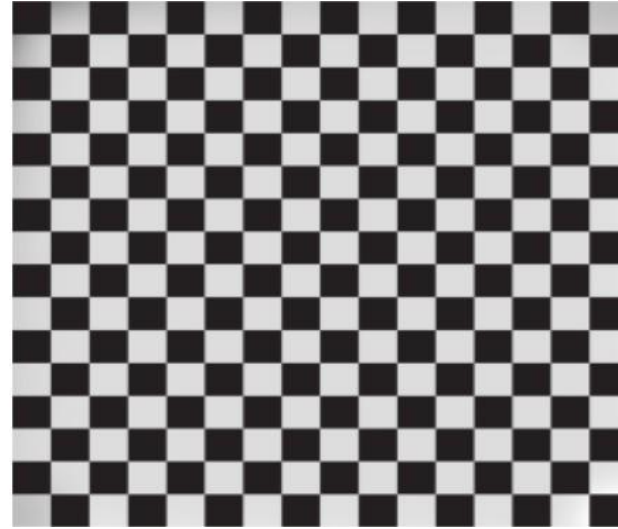


Image shaded by a shading pattern oriented in the -45° direction



Estimate of the shading patterns obtained using lowpass filtering.



Result of dividing (a) by (b). (See Section 9.8 for a morphological approach to shading correction).

Shading Correction

- **Estimating Background:** A Gaussian lowpass filter can be used to extract the general "shading pattern" from an image while ignoring the foreground detail.
- **Correction:** By dividing the original image by this estimated shading pattern, you can produce a corrected image with uniform lighting.



- Readings:

Digital Image Processing, Rafael C. Gonzalez & Richard E woods, Third Ed, Chapter 3;

- Next Class:

Image Sharpening; Linear Filters; Edge Detection using Gradients, Sobel,Canny; Demonstration of Applications in Python

Line detection using Hough transforms; [if possible]